

第11章 并发控制



第11章 并发控制

大纲

- 事务
- 并发控制
- 基于锁的并发控制
- 基于时间戳的并发控制
- 多版本并发控制 (MVCC)

■ 事务 Transaction

```
UPDATE savings_accounts  
SET balance = balance - 500  
WHERE account = 3209;
```

Decrement Savings Account 怎么全执

```
UPDATE checking_accounts  
SET balance = balance + 500  
WHERE account = 3208;
```

Increment Checking Account

```
INSERT INTO journal VALUES  
(journal_seq.NEXTVAL, '1B'  
3209, 3208, 500);
```

Record in Transaction Journal

■ DTL (Data Transaction Language) 事务语句

事务启动: BEGIN, START TRANSACTION

事务提交: COMMIT

➤ 将事务对数据库的修改持久地写到数据库中

事务中止: ROLLBACK

➤ 将事务对数据库的修改全部撤销 (undo) , 就像事务从未执行过

➤ 事务既可以被自己中止, 也可以被 DBMS 中止

■ Demo

```
CREATE TABLE t (  
    id INT PRIMARY KEY,  
    val INT NOT NULL  
);
```

```
SET autocommit=OFF ;
```

Atomicity: 原子性

- all or nothing
- 事务的操作要么全部执行, 要么一个也不执行

Consistency: 一致性

- it looks correct to me
- 如果事务程序正确且启动时处于一致状态, 则事务结束时仍处于一致状态

ACID

Isolation: 隔离性

- as if alone
- 一个事务的执行不受其他事务的干扰

Durability: 持久性

- survive failures
- 事务一旦提交, 它对数据库的修改一定全部持久地写到数据库中

■ 原子性

"All or Nothing"

事务的执行只有两种结局

- 执行完所有操作后提交 $\Rightarrow$ 不破坏原子性
- 执行了部分操作后中止 $\Rightarrow$ 破坏原子性

事务的原子性由 DBMS 保证

- 中止事务时执行过的操作必须撤销 (undo) (第12章: 故障恢复)

■ 持久性

"Survive Failures"

故障导致事务对数据库的修改有 4 种结果

- 提交事务的修改已全部持久存储 ⇒ 不破坏持久性
- 提交事务的修改仅部分持久存储 ⇒ 破坏持久性
- 中止事务的修改有些已持久存储 ⇒ 破坏持久性
- 中止事务的修改未持久存储 ⇒ 不破坏持久性

事务的持久性由 DBMS 保证（第12章：故障恢复）

- 重做（redo）提交事务对数据库的修改
- 撤销（undo）中止事务对数据库的修改

■ 一致性

"It Looks Correct to Me"

事务的一致性由用户保证

➤ 别写错程序

■ 隔离性

"As If Alone"

多个事务的执行有 2 种方式

- 串行执行 ⇒ 不破坏隔离性
- 交叉执行 ⇒ 可能破坏隔离性

事务的隔离性由 DBMS 保证（第11章：并发控制）

- 并发控制：确定多个事务的操作的正确交叉执行顺序

大纲

- 事务
- 并发控制
- 基于锁的并发控制
- 基于时间戳的并发控制
- 多版本并发控制 (MVCC)

调度

■ 基本概念

数据库：固定的数据对象集合

- 数据对象是个抽象概念，可以是属性值、元组、页、关系或数据库
- 数据对象又称数据库元素
- 我们先不考虑数据的插入和删除

事务：数据库对象的读/写操作序列

- 事务可以在数据库上进行很多操作，但 DBMS 只关心事务对数据对象的读/写操作

■ 调度

调度是一个或多个事务的重要操作的序列

Example (调度)

T_1	T_2
READ(A, t)	
	READ(A, s)
t := t + 100	
	s := s * 2
WRITE(A, t)	
	WRITE(A, s)
READ(B, t)	
	READ(B, s)
t := t + 100	
	s := s * 2
WRITE(B, t)	
	WRITE(B, s)

■ 串行调度

如果一个调度中不同事务的操作没有交叉，则该调度是串行调度

Example (串行调度)

T_1	T_2	T_1	T_2
READ(A, t)			READ(A, s)
t := t + 100			s := s * 2
WRITE(A, t)			WRITE(A, s)
READ(B, t)			READ(B, s)
t := t + 100			s := s * 2
WRITE(B, t)			WRITE(B, s)
	READ(A, s)	READ(A, t)	
	s := s * 2	t := t + 100	
	WRITE(A, s)	WRITE(A, t)	
	READ(B, s)	READ(B, t)	
	s := s * 2	t := t + 100	
	WRITE(B, s)	WRITE(B, t)	

■ 非串行调度

不是串行调度的调度称为非串行调度

Example (非串行调度)

T_1	T_2
READ(A, t)	
t := t + 100	
WRITE(A, t)	
	READ(A, s)
	s := s * 2
	WRITE(A, s)
READ(B, t)	
t := t + 100	
WRITE(B, t)	
	READ(B, s)
	s := s * 2
	WRITE(B, s)

T_1	T_2
READ(A, t)	
	READ(A, s)
t := t + 100	
	s := s * 2
WRITE(A, t)	
	WRITE(A, s)
READ(B, t)	
	READ(B, s)
t := t + 100	
	s := s * 2
WRITE(B, t)	
	WRITE(B, s)

■ 调度的正确性

每个事务孤立执行，都会将数据库从一种一致性状态变为另一种一致性状态

Example (The Correctness Principle)

设数据库的一致性约束条件(consistency constraint)为 $A = B$

T_1	A	B	T_2	A	B
	25	25		25	25
READ(A, t)			READ(A, s)		
t := t + 100			s := s * 2		
WRITE(A, t)	125		WRITE(A, s)	50	
READ(B, t)			READ(B, s)		
t := t + 100			s = s * 2		
WRITE(B, t)		125	WRITE(B, s)		50

任意串行调度都能保持数据库的一致性

Example (Correct Serial Schedules)

T_1	T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
WRITE(A, t)		125	
READ(B, t)			
t := t + 100			
WRITE(B, t)			125
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	250	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		250

不同的串行调度可能导致数据库处于不同的最终状态，但都是一致状态

Example (Correct Serial Schedules)

T_1	T_2	A	B
		25	25
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	50	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		50
READ(A, t)			
t := t + 100			
WRITE(A, t)		150	
READ(B, t)			
t := t + 100			
WRITE(B, t)			150

非串行调度可能会破坏数据库的一致性

Example (Incorrect Nonserial Schedules)

T_1	T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
WRITE(A, t)		125	
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	250	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		50
READ(B, t)			
t := t + 100			
WRITE(B, t)			150

■ 异常

非串行调度会导致事务的异常行为，从而破坏数据库的一致性

- 脏写
- 脏读
- 不可重复读
- 幻读

■ 脏写 (Overwriting Uncommitted Data)

在事务 T_2 提交前, T_2 写的 A 值被事务 T_1 修改

Example (脏写)

T_1	T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	50	
WRITE(A, t)		125	
	READ(B, s)		
	s := s * 2		
READ(B, t)			
t := t + 100			
WRITE(B, t)			125
	WRITE(B, s)		50

■ 脏读 (Reading Uncommitted Data)

在事务 T_1 提交前, T_1 写的 A 值被事务 T_2 读取

Example (脏读)

T_1	T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
WRITE(A, t)		125	
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	250	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		50
READ(B, t)			
t := t + 100			
WRITE(B, t)			150

■ 不可重复读

- 事务 T_2 修改了事务 T_1 读过的对象 A 的值, 并且 T_2 提交
- 如果 T_1 再次读 A 的值, 那么它将得到一个不同于前一次读到的 A 值, 即便 T_1 在此期间并未对 A 进行修改

Example (不可重复读)

设一致性约束条件为 $A \geq 0$

T_1	T_2	A
		1
READ(A, t)		
	READ(A, s)	
	$s := s - 1$	
	WRITE(A, s)	0
	COMMIT	
READ(A, t)		
$t := t - 1$		
WRITE(A, t)		-1

■ 幻读

涉及数据的插入和删除，后面再讲

■ 等价调度

如果两个调度在任意数据库实例上的效果都相同，则这两个调度等价

Example (等价调度)

T_1	T_2	A	B	T_1	T_2
		25	25		
READ(A, t)				READ(A, t)	
t := t + 100				t := t + 100	
WRITE(A, t)		125		WRITE(A, t)	
	READ(A, s)			READ(B, t)	
	s := s * 2			t := t + 100	
	WRITE(A, s)	250		WRITE(B, t)	
READ(B, t)					READ(A, s)
t := t + 100					s := s * 2
WRITE(B, t)			125		WRITE(A, s)
	READ(B, s)				READ(B, s)
	s := s * 2				s := s * 2
	WRITE(B, s)		250		WRITE(B, s)

■ 可串行化调度

如果一个调度等价于某串行调度，则该调度是可串行化调度

Example (可串行化调度)

T_1	T_2	A	B	T_1	T_2
		25	25		
READ(A, t) t := t + 100 WRITE(A, t)		125		READ(A, t) t := t + 100 WRITE(A, t)	
	READ(A, s) s := s * 2 WRITE(A, s)	250		READ(B, t) t := t + 100 WRITE(B, t)	
READ(B, t) t := t + 100 WRITE(B, t)			125		READ(A, s) s := s * 2 WRITE(A, s)
	READ(B, s) s := s * 2 WRITE(B, s)		250		READ(B, s) s := s * 2 WRITE(B, s)

■ 可串行化调度的优点

	脏写	脏读	不可重复读	幻读
可串行化	无	无	无	无

程序员无需考虑事务并发的问题，全部由 DBMS 来解决

■ 可串行化调度的缺点

可串行化调度的并发度低

在某些场景下，并发事务不需要严格隔离

- 一个事务对部分对象的修改可以暴露给对其他并发事务
- 弱隔离级别可以提高系统的并发度

隔离级别

■ 隔离级别

SQL-92 规定了并发事务的 4 种隔离级别

- 读未提交 (Read Uncommitted)
- 读提交 (Read Committed)
- 可重复读 (Repeatable Read)
- 可串行化 (Serializable)

在不同隔离级别下，一个事务修改过的对象的值对其他并发事务的可见程度不同

可以在事务开始前设置事务的隔离级别

- SET TRANSACTION ISOLATION LEVEL <isolation-level> ;
- set transaction_isolation = <isolation-level>;

■ 读未提交

未提交事务所做的修改对其他事务可见

Example (Read Uncommitted)

T_1	T_2	A	Variables
		A = 1	
READ(A, t)			t = 1
	READ(A, s)		s = 1
	s := s * 2		s = 2
	WRITE(A, s)	A = 2	
READ(A, x)			x = 2
	COMMIT		
READ(A, y)			y = 2
COMMIT			
READ(A, z)			z = 2

■ 隔离级别

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能

■ 读提交

只有已提交事务所做的修改才对其他事务可见

Example (Read Committed)

T_1	T_2	A	Variables
		A = 1	
READ(A, t)			t = 1
	READ(A, s)		s = 1
	s := s * 2		s = 2
	WRITE(A, s)	A = 2	
READ(A, x)			x = 1
	COMMIT		
READ(A, y)			y = 2
COMMIT			
READ(A, z)			z = 2

■ 隔离级别

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能
READ COMMITTED	无	可能	可能

■ 可重复读

如果一个事务不修改对象 X 的值，则该事务在任何时候读到的 X 值都等于事务启动时读到的 X 值（是 MySQL 默认的隔离级别）

Example (Repeatable Read)

T_1	T_2	A	Variables
		A = 1	
READ(A, t)			t = 1
	READ(A, s)		s = 1
	s := s * 2		s = 2
	WRITE(A, s)	A = 2	
READ(A, x)			x = 1
	COMMIT		
READ(A, y)			y = 1
COMMIT			
READ(A, z)			z = 2

■ 隔离级别

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能
READ COMMITTED	无	可能	可能
REPEATABLE READ	无	无	可能

■ 可串行化

- 无脏读：T 只读取已提交事务所做的更改
- 可重复读：任何其他事务在 T 完成之前都不会更改 T 读取或写入的任何值
- 无幻读：如果 T 基于某些搜索条件读取一组值，那么在 T 完成之前，其他事务不会更改此集合

■ 隔离级别

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能
READ COMMITTED	无	可能	可能
REPEATABLE READ	无	无	可能
SERIALIZABLE	无	无	无

■ Demo

```
CREATE TABLE t (  
    id INT PRIMARY KEY,  
    val INT NOT NULL  
);
```

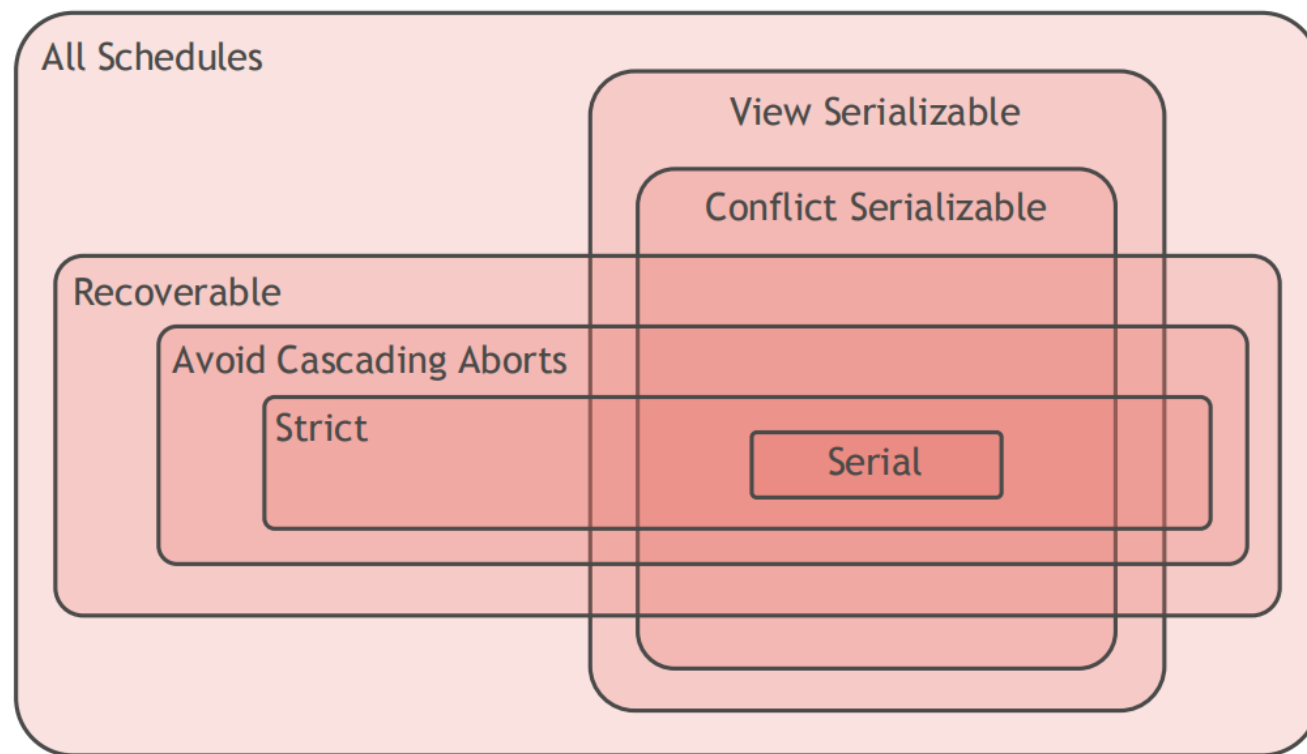
```
SET autocommit=OFF ;
```

并发控制串行化

■ 冲突可串行化

支持可串行化隔离级别的 DBMS 实施的都是冲突可串行化

- 冲突可串行化比一般可串行化的条件更严
- 冲突可串行化更便于在 DBMS 中实施



■ 冲突

两个操作冲突，如果

- 这两个操作属于不同的事务
- 这两个操作涉及相同的对象
- 这两个操作中至少有一个操作是写

■ 写-写冲突

写-写冲突可能导致脏写

Example (写-写冲突)

T_1	T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	50	
WRITE(A, t)		125	
	READ(B, s)		
	s := s * 2		
READ(B, t)			
t := t + 100			
WRITE(B, t)			125
	WRITE(B, s)		50

■ 写-读冲突

写-读冲突可能导致脏读

Example (写-读冲突)

T_1	T_2	A	B
		25	25
READ(A, t)			
t := t + 100			
WRITE(A, t)		125	
	READ(A, s)		
	s := s * 2		
	WRITE(A, s)	250	
	READ(B, s)		
	s := s * 2		
	WRITE(B, s)		50
READ(B, t)			
t := t + 100			
WRITE(B, t)			150

■ 读-写冲突

读-写冲突可能导致不可重复读

Example (读-写冲突)

T_1	T_2	A
		1
READ(A, t)		
	READ(A, s)	
	s := s - 1	
	WRITE(A, s)	0
	COMMIT	
READ(A, t)		
t := t - 1		
WRITE(A, t)		-1

■ 事务的简化表示

对 DBMS 而言，事务中与调度相关的操作只有对象的读和写

Example

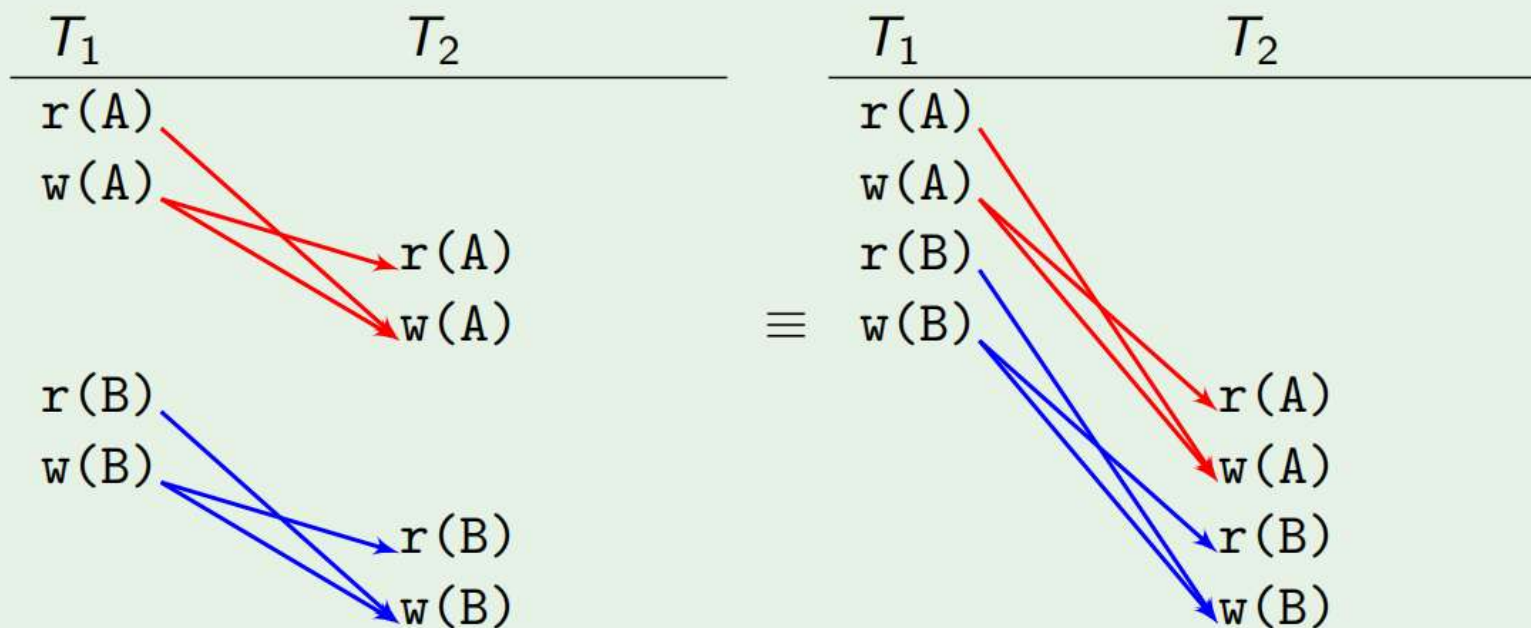
T_1	T_2		T_1	T_2
READ(A, t)			r(A)	
t := t + 100				
WRITE(A, t)			w(A)	
	READ(A, s)			r(A)
	s := s * 2			
	WRITE(A, s)	→		w(A)
READ(B, t)			r(B)	
t := t + 100				
WRITE(B, t)			w(B)	
	READ(B, s)			r(B)
	s := s * 2			
	WRITE(B, s)			w(B)

■ 冲突等价

两个调度冲突等价，如果

- 这两个调度涉及相同事务的相同操作
- 每一对冲突的操作在两个调度中的顺序都相同

Example (冲突等价)



■ 冲突可串行化调度

如果一个调度冲突等价于某个串行调度，则该调度是冲突可串行化调度

Example (冲突可串行化调度)

T_1	T_2		T_1	T_2
$r(A)$			$r(A)$	
$w(A)$			$w(A)$	
	$r(A)$		$r(B)$	
	$w(A)$	$\equiv$	$w(B)$	
$r(B)$				$r(A)$
$w(B)$				$w(A)$
	$r(B)$			$r(B)$
	$w(B)$			$w(B)$

■ 冲突可串行化调度

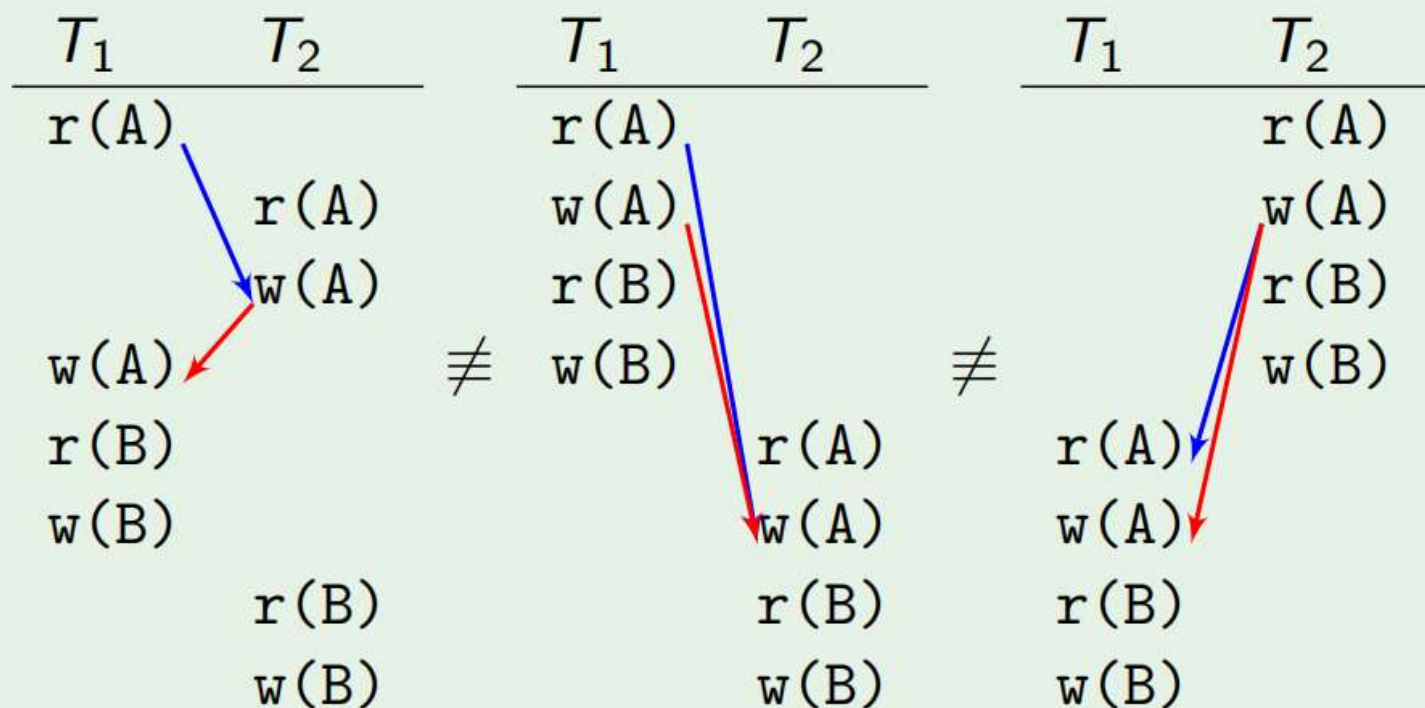
如果通过将调度 S 中不同事务中的非冲突操作交换顺序可以将 S 转换为一个串行调度, 则 S 是一个冲突可串行化调度

Example (冲突可串行化调度)

T_1	T_2		T_1	T_2		T_1	T_2		T_1	T_2
$r(A)$			$r(A)$			$r(A)$			$r(A)$	
$w(A)$			$w(A)$			$w(A)$			$w(A)$	
	$r(A)$			$r(A)$		$r(B)$			$r(B)$	
	$w(A)$	$\equiv$	$r(B)$		$\equiv$		$r(A)$	$\equiv$		$r(A)$
$r(B)$				$w(A)$			$w(A)$		$w(B)$	
$w(B)$			$w(B)$			$w(B)$				$w(A)$
	$r(B)$			$r(B)$			$r(B)$			$r(B)$
	$w(B)$			$w(B)$			$w(B)$			$w(B)$

■ 非冲突可串行化调度

Example (非冲突可串行化调度)



■ 冲突可串行化测试

第 1 步：将调度 S 表示为优先图

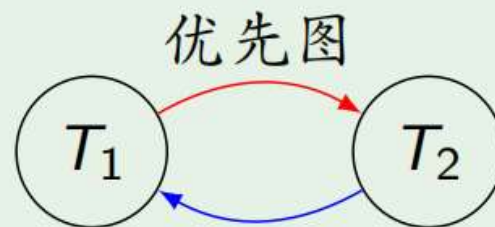
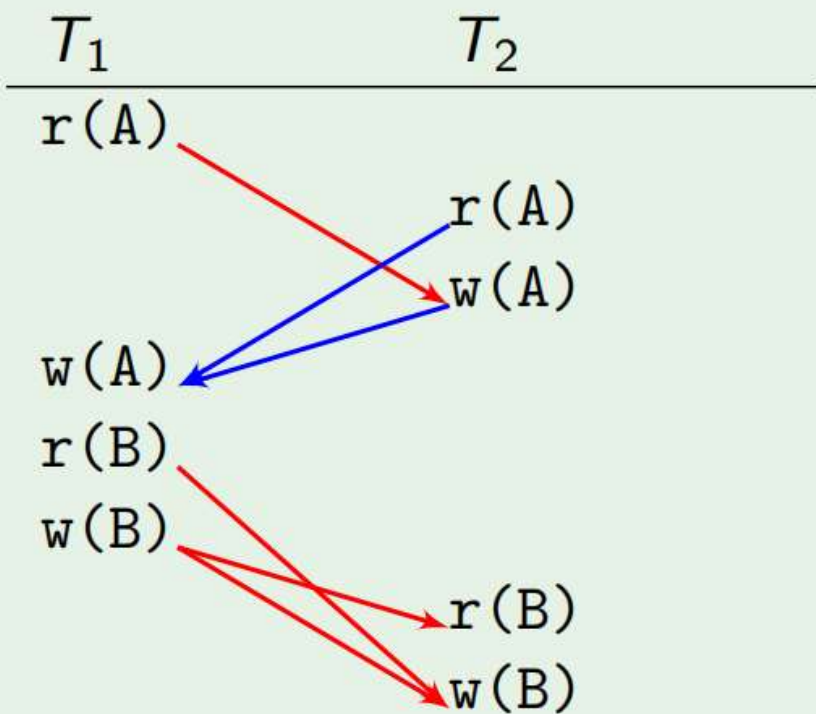
- 每个顶点代表 S 中的一个事务
- 从事务 T_i 到事务 T_j 有一条有向边，如果 T_i 的某个操作 o_i 和 T_j 的某个操作 o_j 冲突，并且 o_i 在 S 中先于 o_j

第 2 步： S 是冲突可串行化调度，当且仅当其优先图没有环

- 优先图顶点的任意拓扑序表示了一个与 S 冲突等价的串行调度

■ 冲突可串行化测试例 1

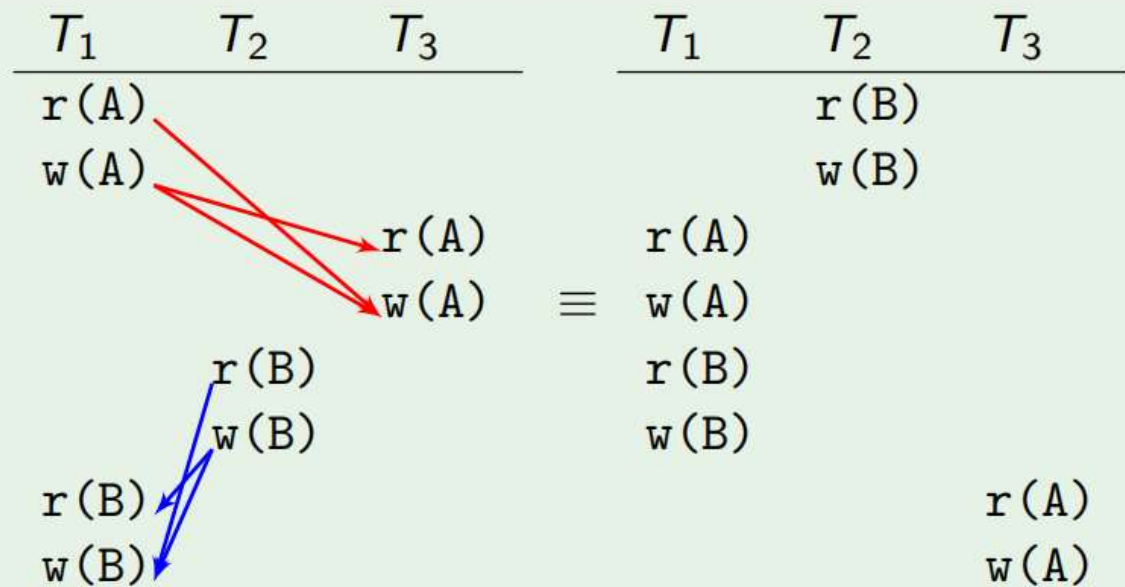
Example (冲突可串行化测试)



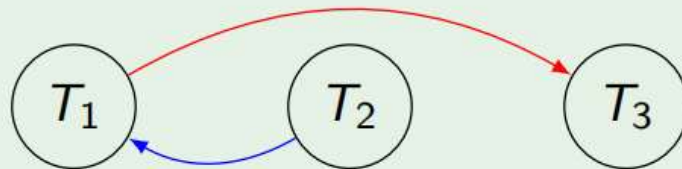
该调度不是冲突可串行化调度

■ 冲突可串行化测试例 2

Example (冲突可串行化测试)



优先图

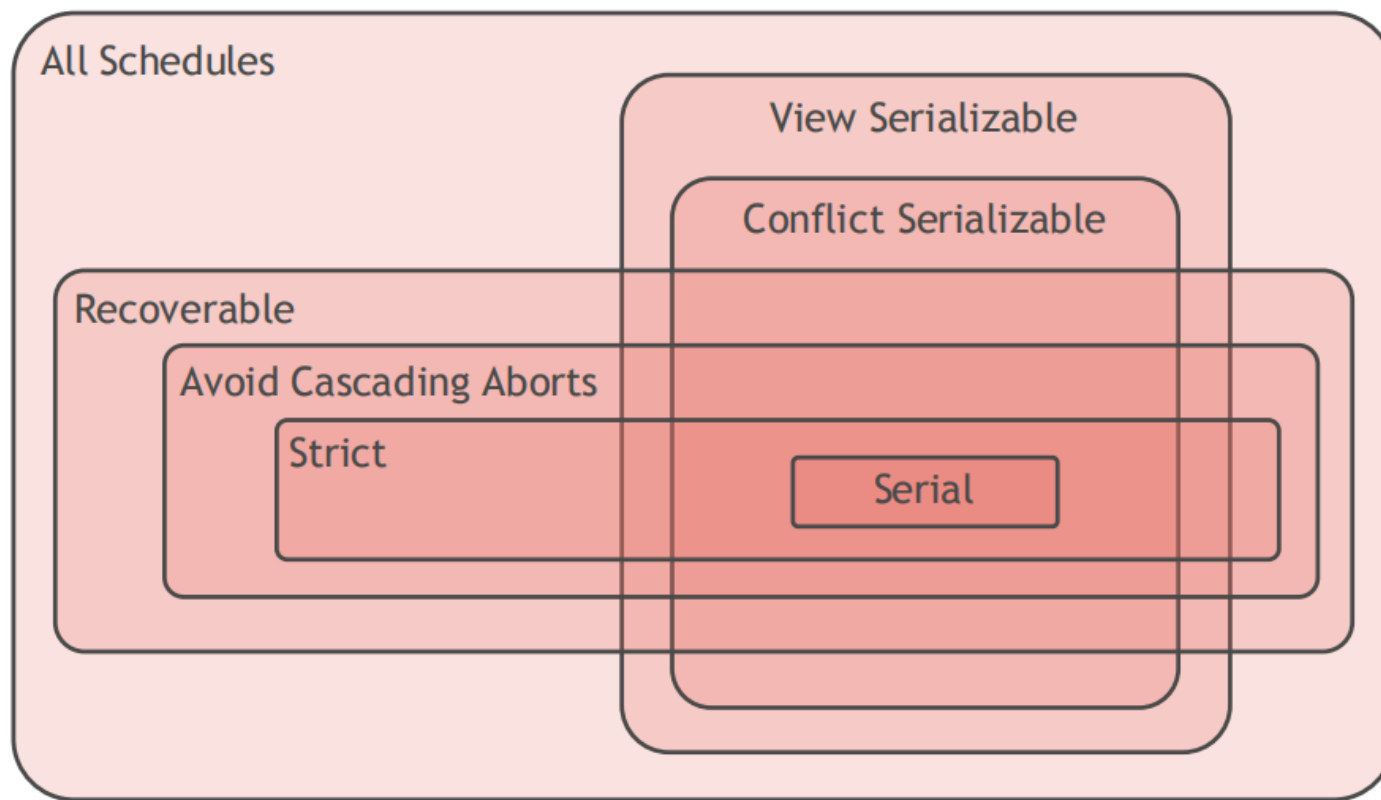


该调度是冲突可串行化调度

■ 视图可串行化

视图可串行化是比冲突可串行化更弱的概念.

- 测试和实施视图可串行化很难
- 没有 DBMS 实施视图可串行化



■ 视图可串行化调度

两个调度 S_1 和 S_2 视图等价, 如果

- 如果 S_1 中事务 T_1 读了对象 A 的初始值, 则 S_2 中 T_1 也读了 A 的初始值
- 如果 S_1 中事务 T_1 读了事务 T_2 修改过的 A 值, 则 S_2 中 T_1 也读了 T_2 修改过的 A 值
- 如果 S_1 中事务 T_1 写了 A 的最终值, 则 S_2 中 T_1 也写了 A 的最终值

如果一个调度视图等价于某个串行调度, 则该调度是视图可串行化调度

Example (视图可串行化调度)

T_1	T_2	T_3		T_1	T_2	T_3
$r(A)$				$r(A)$		
	$w(A)$		$\equiv_{view}$	$w(A)$		
$w(A)$					$w(A)$	
		$w(A)$				$w(A)$

■ 冲突可串行化 vs 视图可串行化

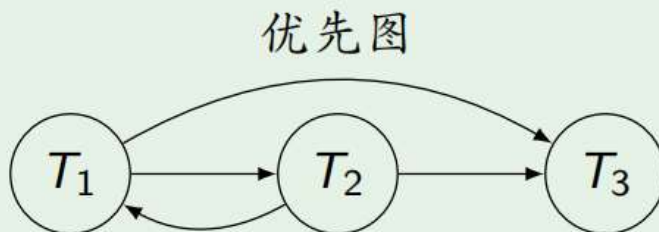
冲突可串行化调度一定是视图可串行化调度

视图可串行化调度不一定是冲突可串行化调度

➤ 视图可串行化但非冲突可串行化的调度一定包含盲写

Example

T_1	T_2	T_3		T_1	T_2	T_3
$r(A)$				$r(A)$		
	$w(A)$		$\equiv_{view}$	$w(A)$		
$w(A)$					$w(A)$	
		$w(A)$				$w(A)$



该调度是视图可串行化调度，但不是冲突可串行化调度

■ 并发控制协议

并发控制协议用于对并发事务实施正确的（运行时）调度，而无需预先确定整个（静态）调度

悲观并发控制

- 假定冲突很多
- 不允许任何冲突发生

乐观并发控制

- 假定冲突很少
- 冲突发生了，再去解决

■ 并发控制协议

方法 1：基于锁的并发控制

方法 2：时间戳定序的并发控制

方法 3：多版本并发控制

大纲

- 事务
- 并发控制
- 基于锁的并发控制
- 基于时间戳的并发控制
- 多版本并发控制 (MVCC)

■ 锁

用锁来保护数据库对象

- 事务 T_i 只有获得了对象 A 的锁，才能读或写 A
- 如果事务 T_i 请求了对象 A 的锁，但并未获得，则 T_i 开始等待，直至获得 A 的锁为止
- 如果事务 T_i 已经获得了对象 A 的锁，则在 T_i 完成对 A 的操作后， T_i 必须释放 A 的锁

■ 使用锁的并发事务执行

- LOCK(A): 请求对 A 加锁
- UNLOCK(A): 释放 A 的锁

Example

T_1	T_2	Actions of the Scheduler
LOCK(A)		The lock on A is granted to T_1
r(A)		
	LOCK(A)	
w(A)		
r(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
	UNLOCK(A)	

■ 使用锁的并发事务执行

- LOCK(A): 请求对 A 加锁
- UNLOCK(A): 释放 A 的锁

Example

T_1	T_2	Actions of the Scheduler
LOCK(A)		The lock on A is granted to T_1
r(A)		
	LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
	UNLOCK(A)	

■ 使用锁的并发事务执行

- LOCK(A): 请求对 A 加锁
- UNLOCK(A): 释放 A 的锁

Example

T_1	T_2	Actions of the Scheduler
LOCK(A)		The lock on A is granted to T_1
r(A)		
	LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		The lock on A is released
	r(A)	
	w(A)	
	UNLOCK(A)	

■ 使用锁的并发事务执行

- LOCK(A): 请求对 A 加锁
- UNLOCK(A): 释放 A 的锁

Example

T_1	T_2	Actions of the Scheduler
LOCK(A)		The lock on A is granted to T_1
r(A)		
	LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		The lock on A is released
	r(A)	The lock on A is granted to T_2
	w(A)	
	UNLOCK(A)	

■ 使用锁的并发事务执行

- LOCK(A): 请求对 A 加锁
- UNLOCK(A): 释放 A 的锁

Example

T_1	T_2	Actions of the Scheduler
LOCK(A)		The lock on A is granted to T_1
r(A)		
	LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		The lock on A is released
	r(A)	The lock on A is granted to T_2
	w(A)	
	UNLOCK(A)	The lock on A is released

■ 锁的类型

共享锁/S 锁

- 事务 T_i 只有获得了对象 A 的共享锁, 才能读 A

互斥锁/X 锁

- 事务 T_i 只有获得了对象 A 的互斥锁, 才能写 A
- T_i 获得了 A 的互斥锁后, 亦可读 A

■ 锁的相容性

如果对象 A 上有事务 T_i 加的共享锁，则事务 T_i 还可以对 A 加共享锁，但不可
以对 A 加互斥锁

➤ 否则产生读-写冲突

如果对象 A 上有事务 T_i 加的互斥锁，则事务 T_i 对 A 既不能加共享锁，也不能
加互斥锁

➤ 否则产生写-读冲突或写-写冲突

相容矩阵(Compatibility Matrix)

		请求加锁的类型	
		共享锁	互斥锁
已加锁 的类型	共享锁	Yes	No
	互斥锁	No	No

■ 使用共享锁和互斥锁的并发事务执行

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	
w(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

■ 使用共享锁和互斥锁的并发事务执行

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

■ 使用共享锁和互斥锁的并发事务执行

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

■ 使用共享锁和互斥锁的并发事务执行

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	The exclusive lock on A is granted to T_2
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

■ 使用共享锁和互斥锁的并发事务执行

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	The exclusive lock on A is granted to T_2
	w(A)	
S-LOCK(A)		Denied
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

■ 使用共享锁和互斥锁的并发事务执行

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	The exclusive lock on A is granted to T_2
	w(A)	
S-LOCK(A)		Denied
	UNLOCK(A)	The exclusive lock on A is released
r(A)		
UNLOCK(A)		

■ 使用共享锁和互斥锁的并发事务执行

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	The exclusive lock on A is granted to T_2
	w(A)	
S-LOCK(A)		Denied
	UNLOCK(A)	The exclusive lock on A is released
r(A)		The shared lock on A is granted to T_1
UNLOCK(A)		

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
$r(A)$		
	X-LOCK(A)	Denied
$w(A)$		
UNLOCK(A)		The exclusive lock on A is released
	$r(A)$	The exclusive lock on A is granted to T_2
	$w(A)$	
S-LOCK(A)		Denied
	UNLOCK(A)	The exclusive lock on A is released
$r(A)$		The shared lock on A is granted to T_1
UNLOCK(A)		The shared lock on A is released

Example

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
$r(A)$		
	X-LOCK(A)	Denied
$w(A)$		
UNLOCK(A)		The exclusive lock on A is released
	$r(A)$	The exclusive lock on A is granted to T_2
	$w(A)$	
S-LOCK(A)		Denied
	UNLOCK(A)	The exclusive lock on A is released
$r(A)$		The shared lock on A is granted to T_1
UNLOCK(A)		The shared lock on A is released

该调度不是冲突可串行化调度(如何证明?)

■ Demo

```
CREATE TABLE t (  
    id INT PRIMARY KEY,  
    val INT NOT NULL  
);
```

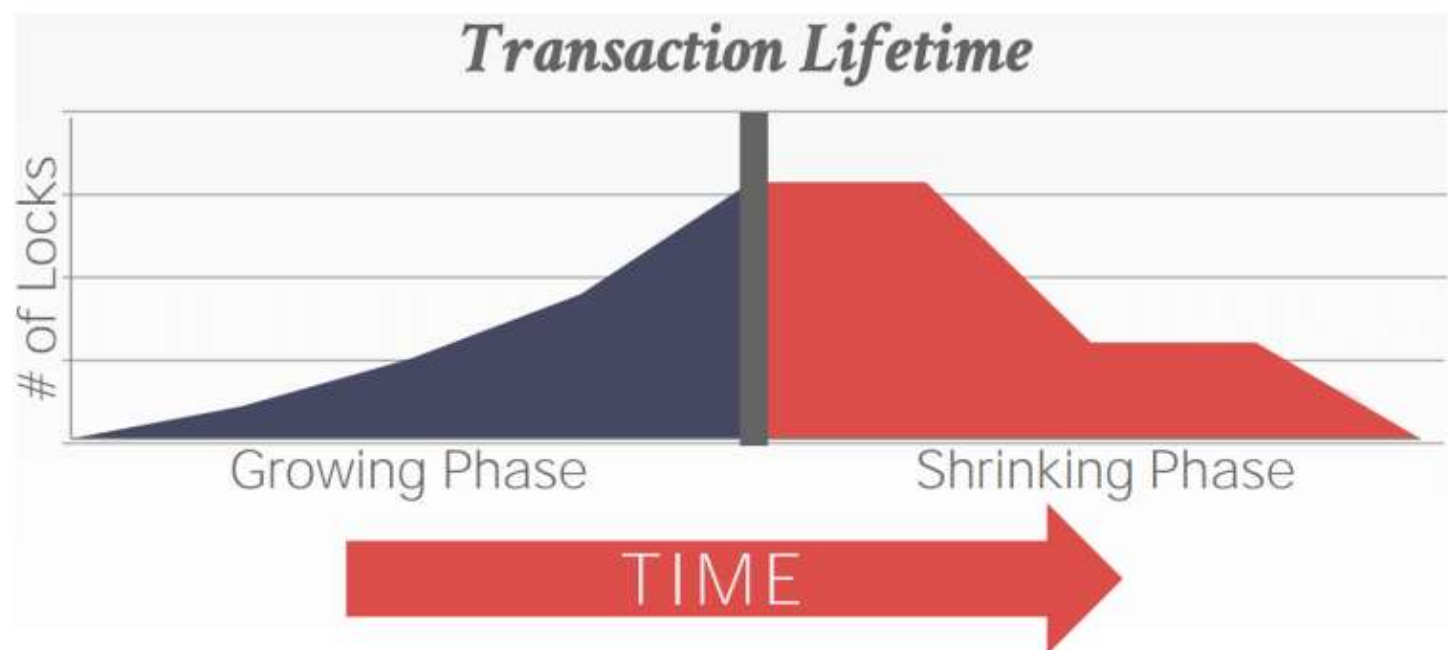
```
SET autocommit=OFF ;
```

两段锁

■ 两阶段锁

每个事务的执行分为两个阶段

- 增长阶段：事务向锁管理器请求需要的锁
- 萎缩阶段：事务释放它获得的锁，但不能再请求加锁



■ 两阶段锁

Example (2PL)

T_1	T_2	Actions of the Scheduler
X-LOCK(A) r(A) w(A) r(A) UNLOCK(A)	 X-LOCK(A) r(A) w(A) UNLOCK(A)	The exclusive lock on A is granted to T_1

■ 两阶段锁

Example (2PL)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
	UNLOCK(A)	

■ 两阶段锁

Example (2PL)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	
	w(A)	
	UNLOCK(A)	

■ 两阶段锁

Example (2PL)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	The exclusive lock on A is granted to T_2
	w(A)	
	UNLOCK(A)	

■ 两阶段锁

Example (2PL)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
$r(A)$		
	X-LOCK(A)	Denied
$w(A)$		
$r(A)$		
UNLOCK(A)		The exclusive lock on A is released
	$r(A)$	The exclusive lock on A is granted to T_2
	$w(A)$	
	UNLOCK(A)	The exclusive lock on A is released

■ 两阶段锁

Example (2PL)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	X-LOCK(A)	Denied
w(A)		
r(A)		
UNLOCK(A)		The exclusive lock on A is released
	r(A)	The exclusive lock on A is granted to T_2
	w(A)	
	UNLOCK(A)	The exclusive lock on A is released

该调度是冲突可串行化调度(如何证明?)

■ 两阶段锁的性质

优点： 2PL 能够保证冲突可串行化

缺点 1： 2PL 面临着级联中止的问题

缺点 2： 2PL 可能导致死锁

■ 级联中止

一个事务中止可能会导致其他事务级联中止

Example (级联中止)

T_1	T_2
X-LOCK(A)	
X-LOCK(B)	
r(A)	X-LOCK(A)
w(A)	
UNLOCK(A)	
	r(A)
	w(A)
r(B)	
w(B)	
ABORT	
	r(A)

- 2PL 允许该调度
- T_1 中止时, T_2 也要中止(级联中止), 否则 T_1 修改过的信息将通过 T_2 泄露给外界

强两阶段锁

■ 严格调度

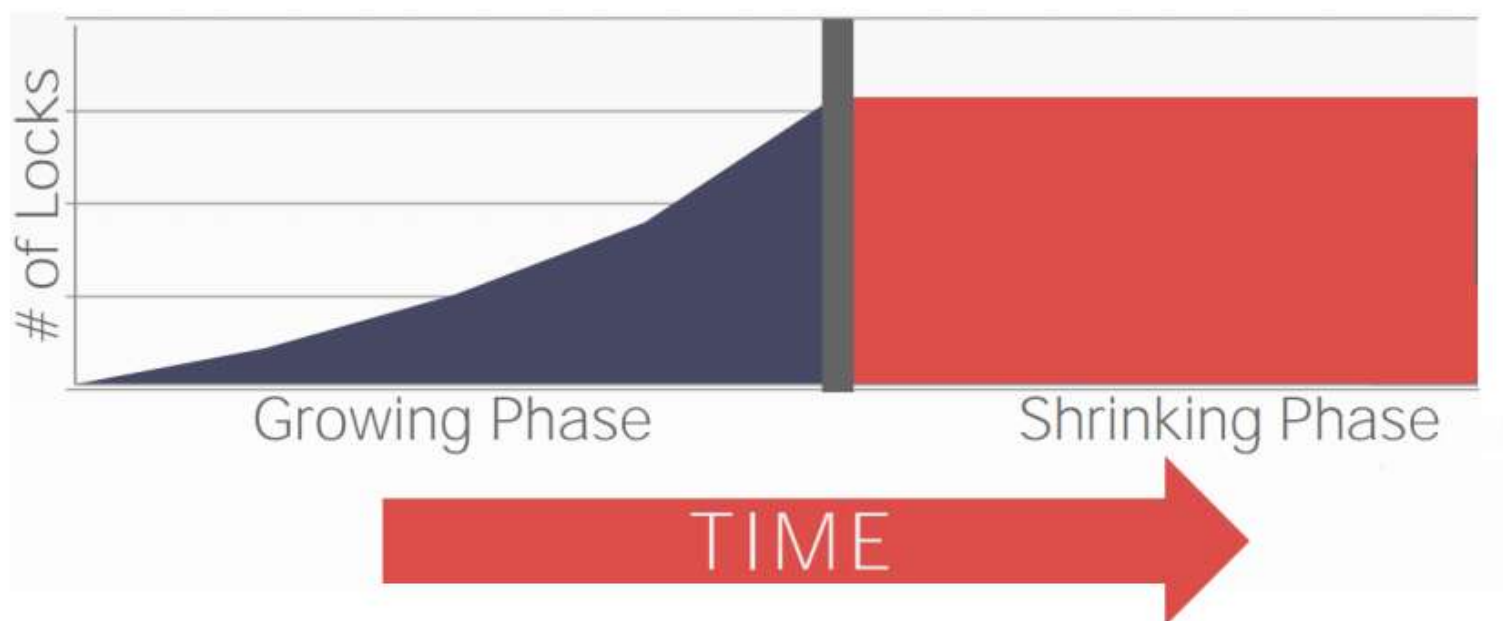
一个调度被称为严格调度，如果该调度中任意事务在结束前由它写入的数据库对象的值没有被其他事务读过或修改过

➤ 严格调度不会引发级联中止

■ 强两阶段锁

- 增长阶段：与 2PL 的增长阶段相同
- 萎缩阶段：当事务结束时，释放它获得的全部的锁

SS2PL 保证生成严格的冲突可串行化调度，因而不会产生级联中止

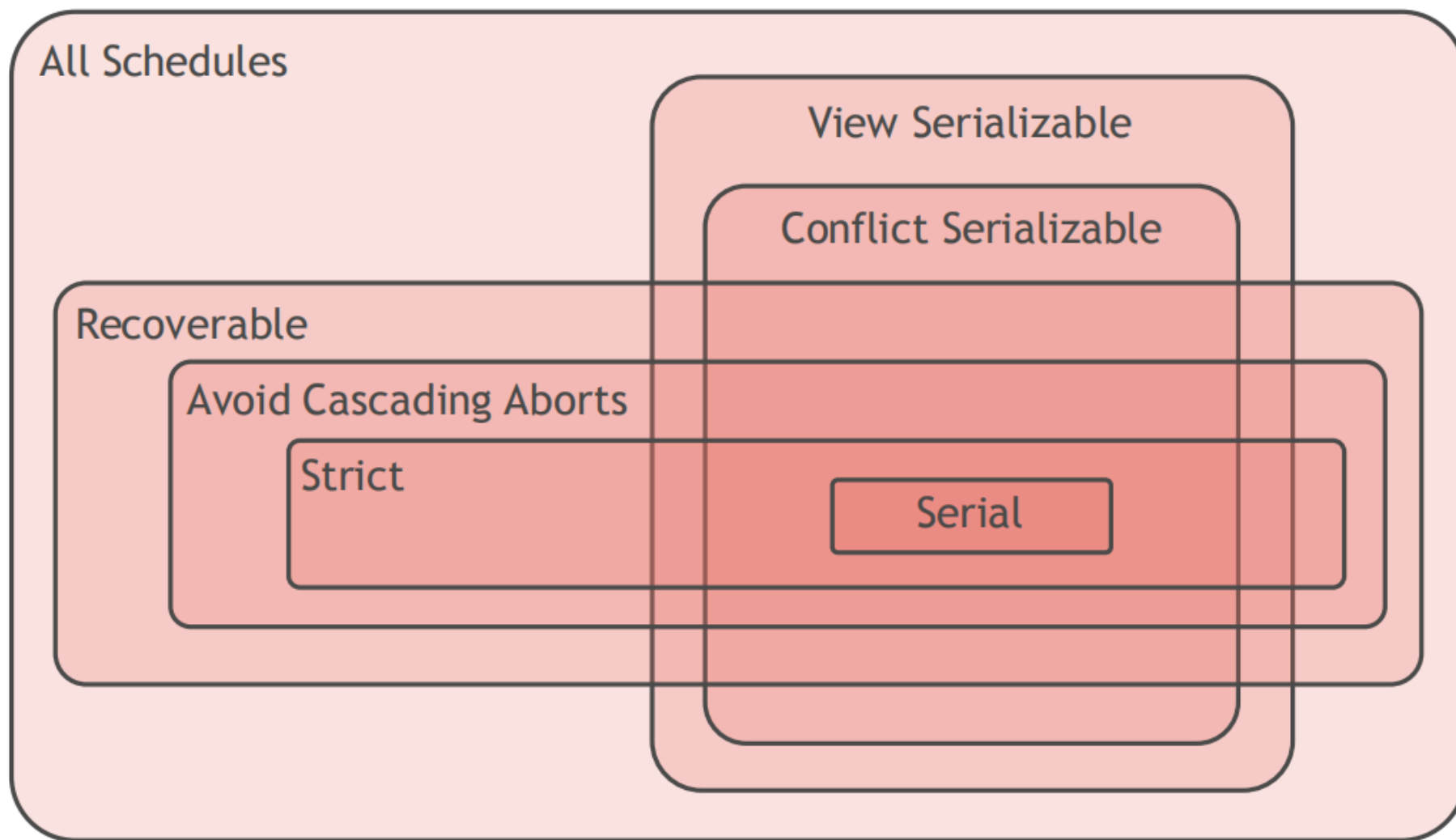


■ SS2PL

Example (SS2PL)

T_1	T_2
X-LOCK(A)	
X-LOCK(B)	
r(A)	X-LOCK(A)
w(A)	
r(B)	
w(B)	
UNLOCK(A)	
UNLOCK(B)	
ABORT	
	r(A)
	w(A)
	r(A)

■ 调度的类别



死锁

■ 死锁

一组事务形成死锁，如果每个事务都在等待其他事务释放锁

Example (死锁)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	S-LOCK(B)	
	r(B)	
	S-LOCK(A)	
w(A)		
X-LOCK(B)		
⋮	⋮	

■ 死锁

一组事务形成死锁，如果每个事务都在等待其他事务释放锁

Example (死锁)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
$r(A)$		
	S-LOCK(B)	The shared lock on B is granted to T_2
	$r(B)$	
	S-LOCK(A)	
$w(A)$		
X-LOCK(B)		
$\vdots$	$\vdots$	

■ 死锁

一组事务形成死锁，如果每个事务都在等待其他事务释放锁

Example (死锁)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
r(A)		
	S-LOCK(B)	The shared lock on B is granted to T_2
	r(B)	
	S-LOCK(A)	Denied!
w(A)		
X-LOCK(B)		
⋮	⋮	

■ 死锁

一组事务形成死锁，如果每个事务都在等待其他事务释放锁

Example (死锁)

T_1	T_2	Actions of the Scheduler
X-LOCK(A)		The exclusive lock on A is granted to T_1
$r(A)$		
	S-LOCK(B)	The shared lock on B is granted to T_2
	$r(B)$	
	S-LOCK(A)	Denied!
$w(A)$		
X-LOCK(B)		Denied!
$\vdots$	$\vdots$	
		形成死锁!

■ 死锁的处理

措施 1：死锁检测

- DBMS 检测死锁是否发生
- 如果发生了死锁，则采取办法解除死锁

措施 2：死锁预防

- 预防死锁的发生

■ 死锁检测

方法 1：超时检测

- 如果在给定的时间内没有任何事务执行，则认为死锁发生

方法 2：等待图检测

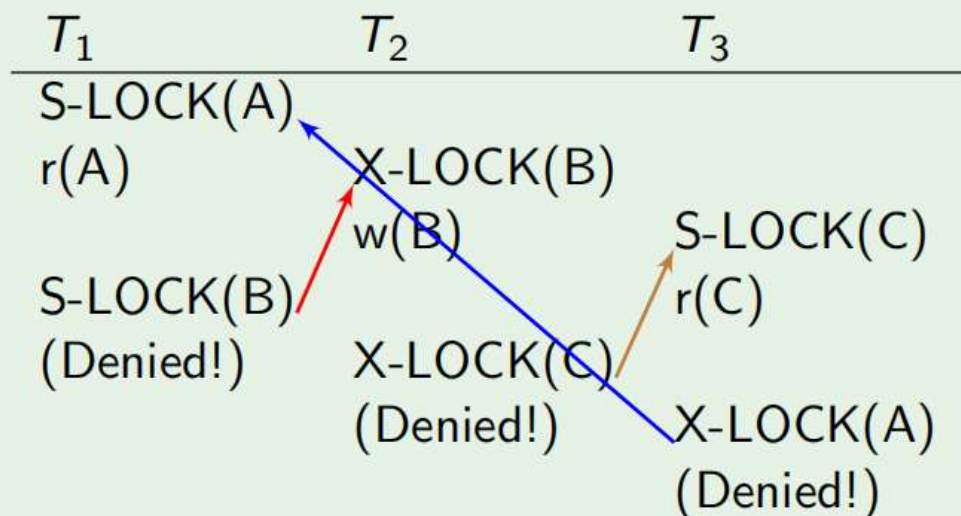
- DBMS 用等待图表示事务关于锁的等待关系
- DBMS 定期检查等待图中是否存在环
- 如果等待图中有环，则死锁发生

■ 等待图

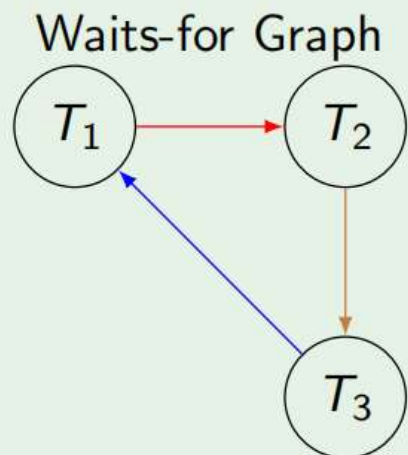
- 顶点代表事务
- 如果事务 T_i 正在等待事务 T_j 释放锁，则存在从 T_i 到 T_j 的有向边

事务产生死锁当且仅当等待图中有环(cycle)

Example (等待图)



T_1, T_2, T_3 产生死锁

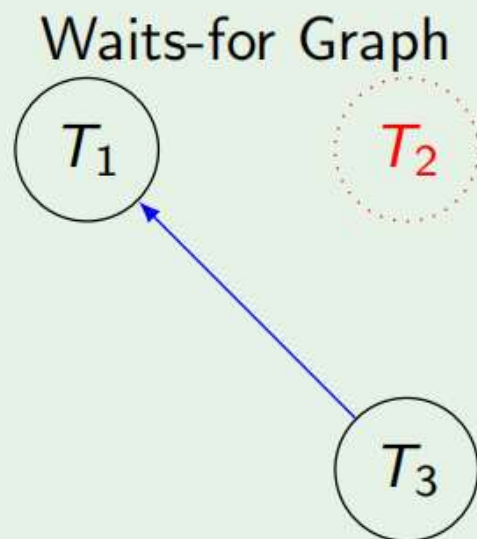


■ 死锁解除

从等待图的环中选择一个事务作为“牺牲品”，中止该事务

Example (死锁解除)

T_1	T_2	T_3
S-LOCK(A) r(A)	X-LOCK(B) w(B)	S-LOCK(C) r(C)
S-LOCK(B) (Denied!)	X-LOCK(C) (Denied!)	X-LOCK(A) (Denied!)
ABORT		



■ 死锁解除

选择“牺牲品”时需要考虑多种因素

- 事务的年龄/启动时间
- 事务的进度(已执行的查询数量)
- 事务获得的锁的数量
- 需要级联中止的事务数量

还要考虑事务“被牺牲”的次数，防止事务“饿死”

■ 死锁预防

当事务启动时，DBMS 为事务分配一个唯一且固定的优先级

➤ 开始得越早，优先级越高

当事务 T_i 请求事务 T_j 拥有的锁而无法获得时，DBMS 根据 T_i 和 T_j 的优先级来裁决如何处理 T_i 和 T_j ，可采用下面 2 条规则之一来预防

规则1: Wait-Die (“Old Waits for Young”)

- 如果 T_i 比 T_j 的优先级高，则 T_i 等待
- 否则， T_i 中止

规则2: Wound-Wait (“Young Waits for Old”)

- 如果 T_i 比 T_j 的优先级高，则 T_j 中止
- 否则， T_i 等待

事务中止并重启后，其优先级保持不变

■ 死锁预防

Example (死锁预防)

T_1 的优先级高于 T_2 的优先级			
T_1	T_2	Wait-Die	Wound-Wait
X-LOCK(A)	X-LOCK(A)	T_1 waits	T_2 aborts
T_1	T_2	Wait-Die	Wound-Wait
X-LOCK(A)	X-LOCK(A)	T_2 aborts	T_2 waits

多粒度锁

■ 锁的效率问题

锁越多，管理锁的开销越大

- 一个事务访问 1 亿条元组，就需要加 1 亿把锁

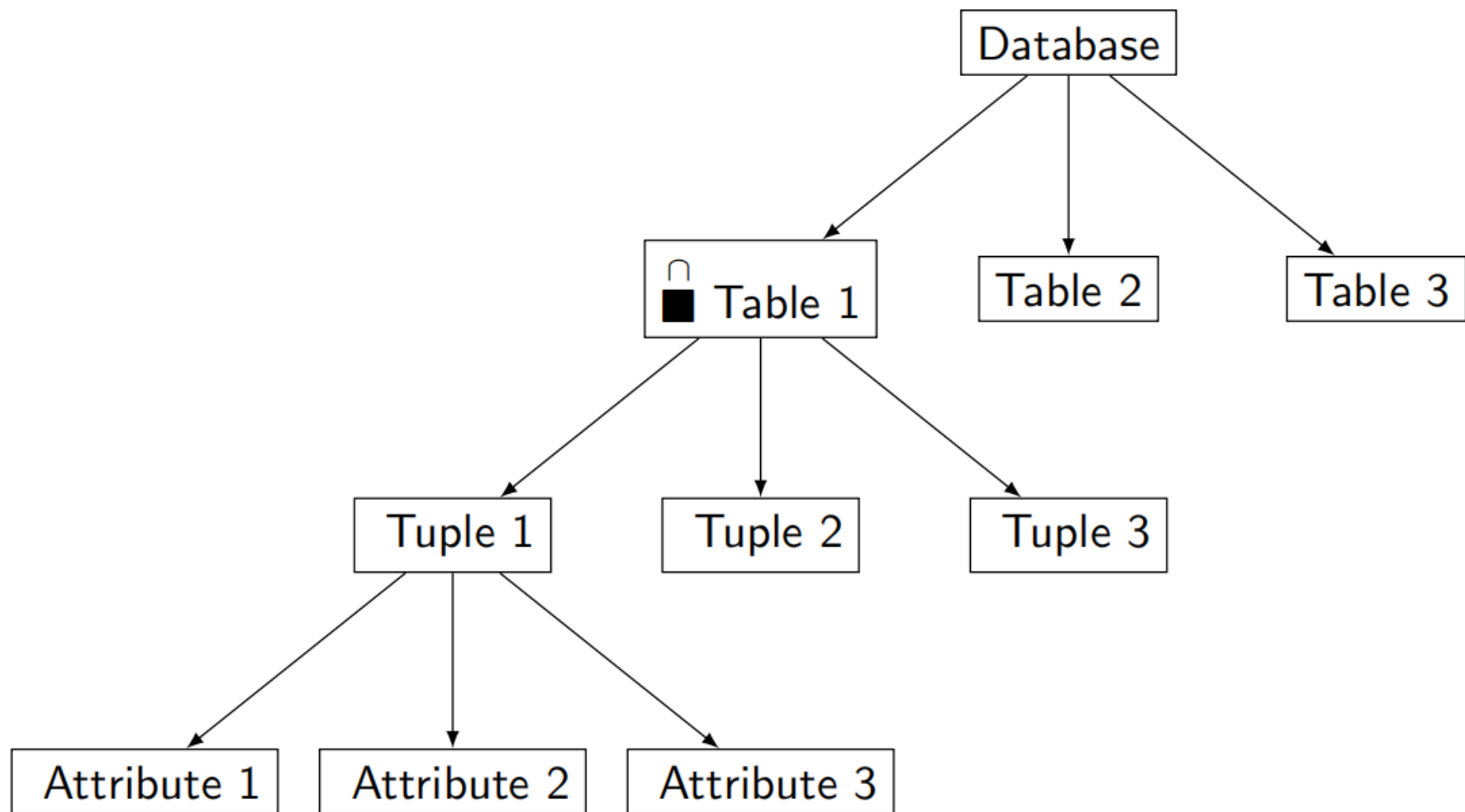
在不合适的粒度上加锁会降低事务并发度

- 一个事务只需访问 1 条元组，却在整个关系上加锁

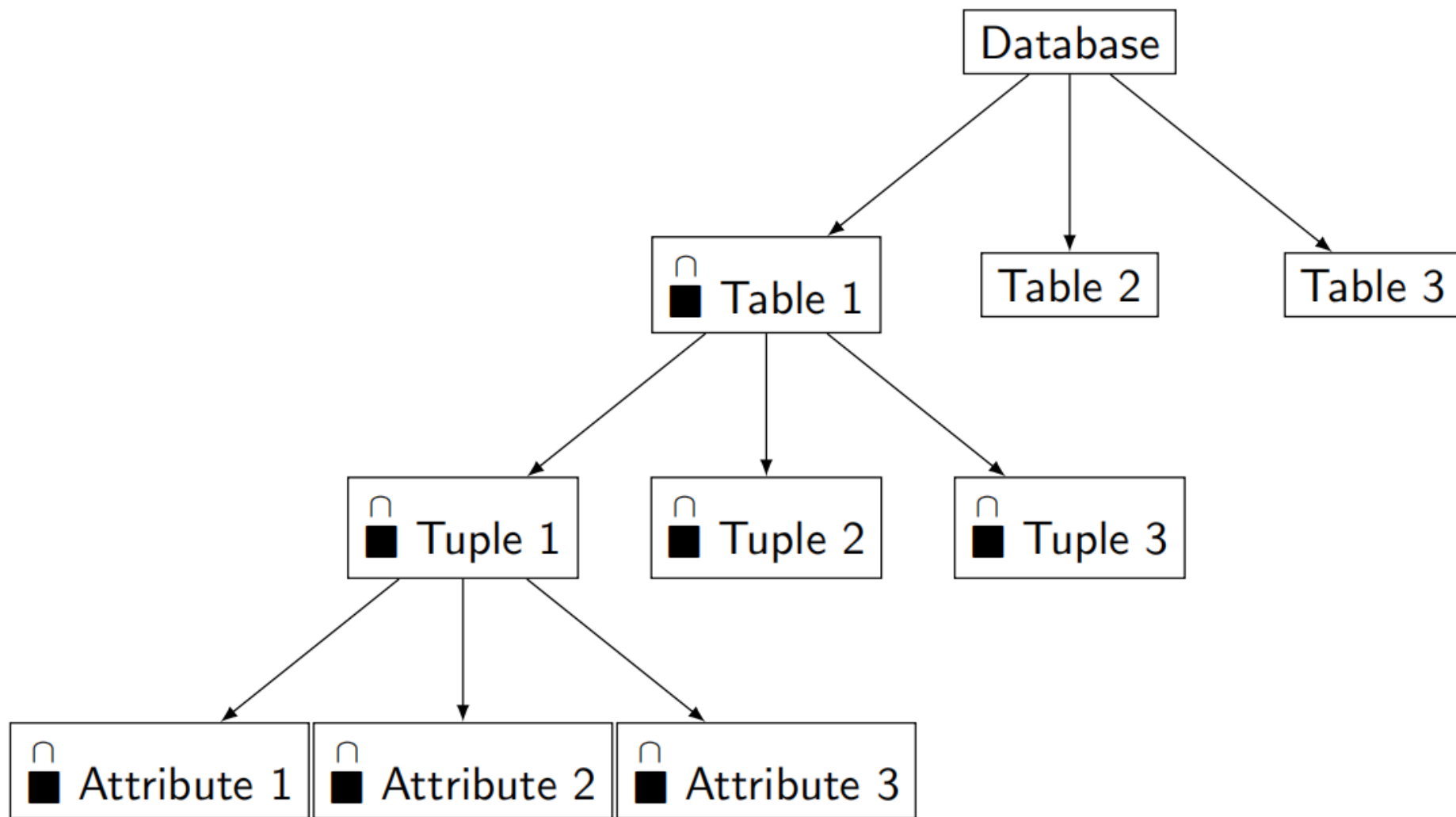
提高锁管理的效率

- 尽可能少加锁
- 在合适的粒度上加锁

■ 锁的粒度



■ 锁的粒度



■ 意向锁

■^{IS} 意向共享锁(Intension-Shared Locks)/IS锁(IS-Locks)

- 对一个对象加IS锁表示要对该对象的某个(些)后裔加S锁

■^{IX} 意向互斥锁(Intension-Exclusive Locks)/IX锁(IX-Locks)

- 对一个对象加IX锁表示要对该对象的某个(些)后裔加X锁或S锁

■^{SIX} 共享意向互斥锁(Shared Intension-Exclusive Locks)/SIX锁(SIX-Locks)

- 对一个对象加SIX锁表示要对该对象及其所有后裔加S锁，并且对该对象的某个(些)后裔加X锁

■ 相容矩阵

		请求锁类型				
		IS	IX	S	SIX	X
拥有锁类型	IS	Yes	Yes	Yes	Yes	No
	IX	Yes	Yes	No	No	No
	S	Yes	No	Yes	No	No
	SIX	Yes	No	No	No	No
	X	No	No	No	No	No

■ 多粒度锁协议

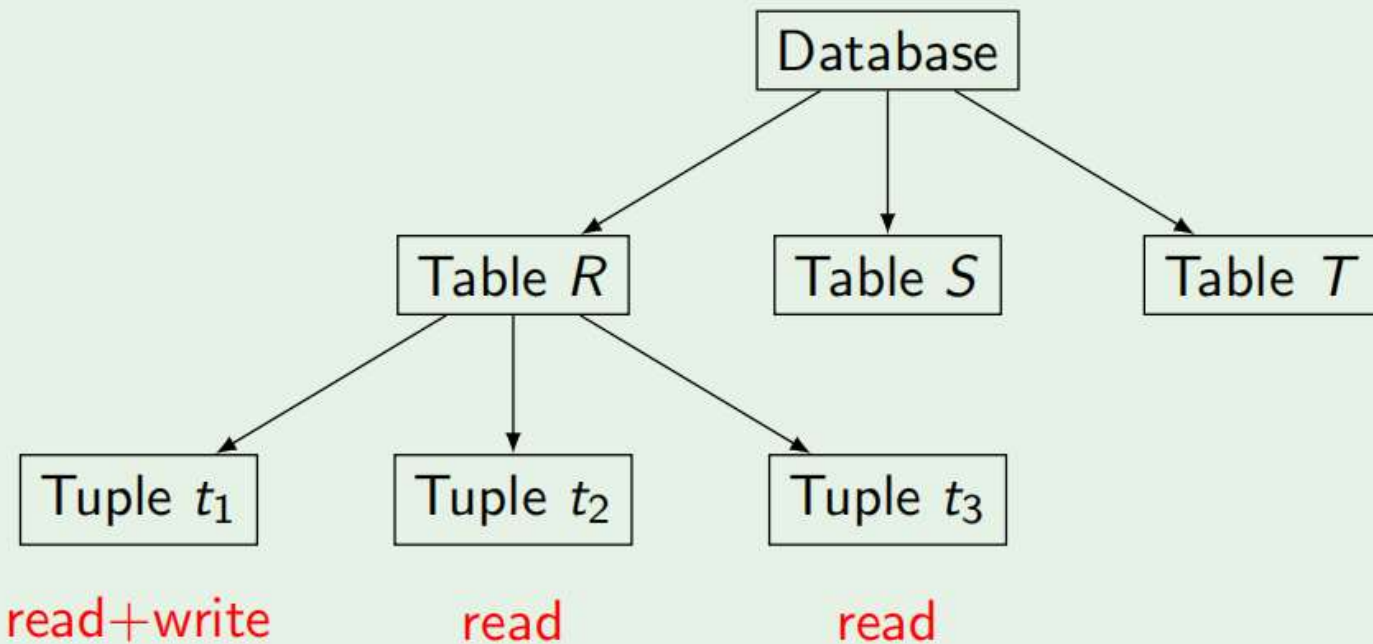
任何事务都要服从下列规则

- 从最高级别对象开始加锁，加锁过程自顶向下
- 对一个对象加 IS 或 S 锁之前，必须先获得其父亲对象的 IS 锁
- 事务对一个对象加 IX、SIX 或 X 锁之前，必须先获得其父亲对象的 IX 锁
- 解锁过程自底向上

■ 多粒度锁协议

Example (多粒度锁协议)

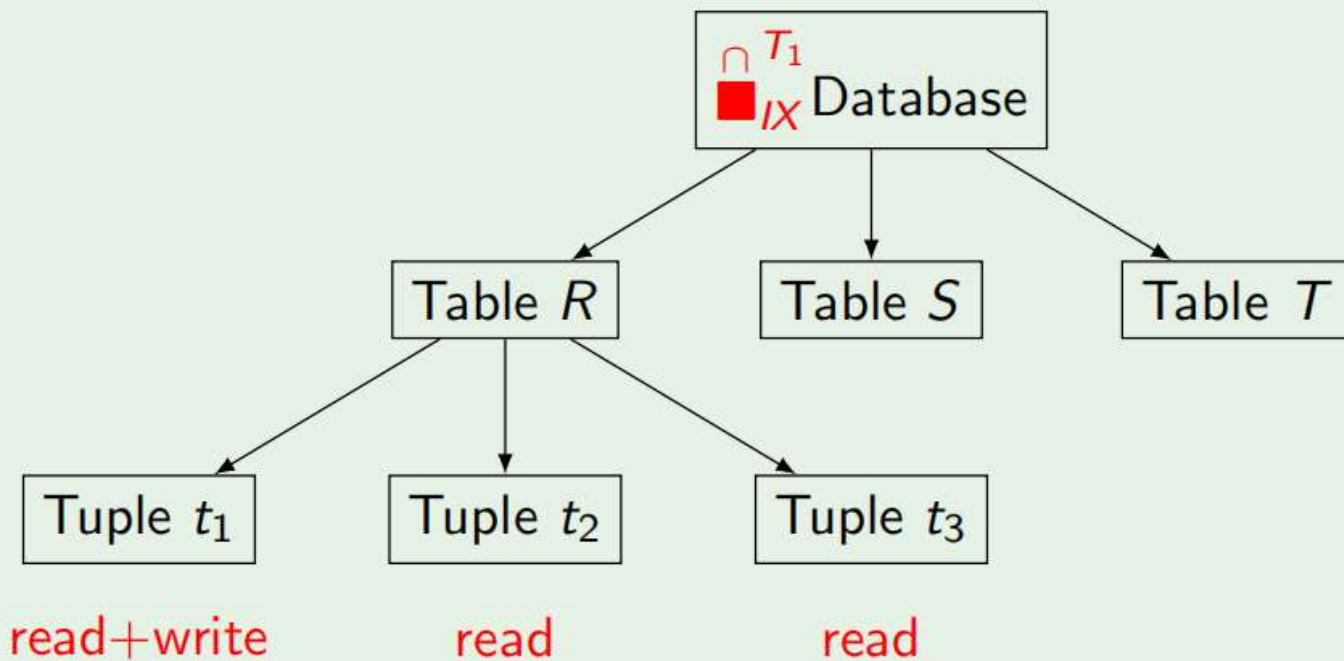
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

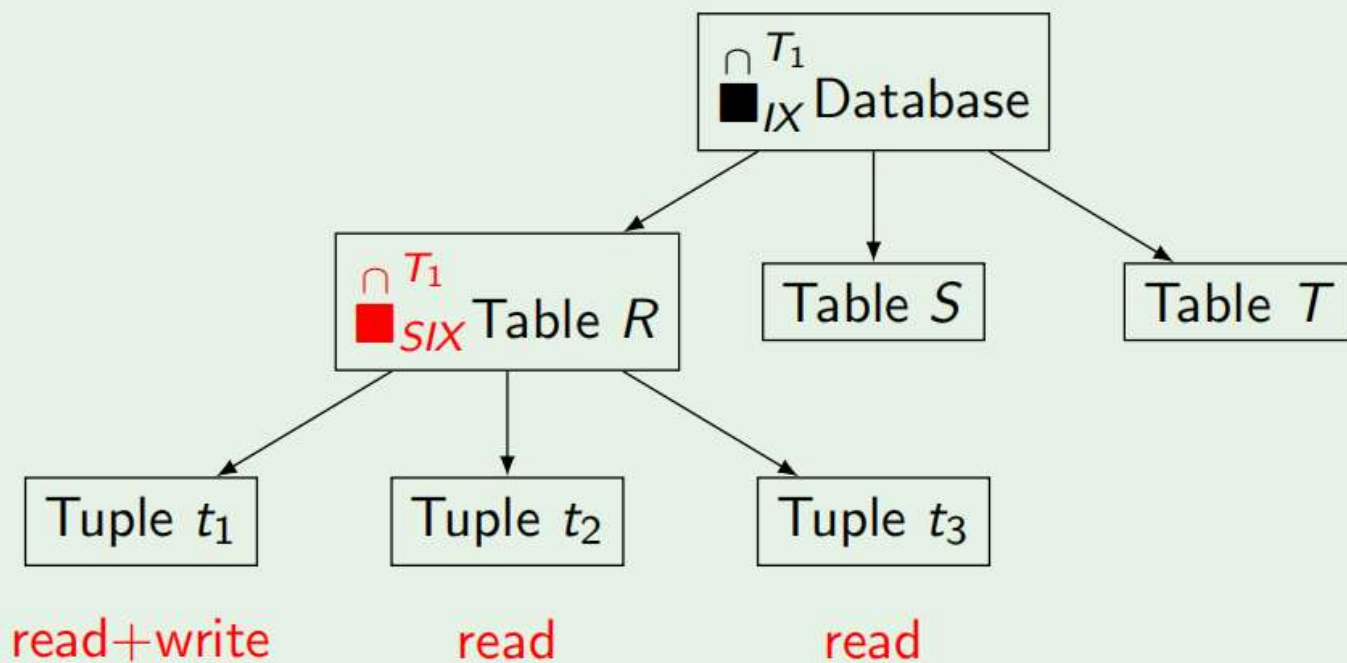
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

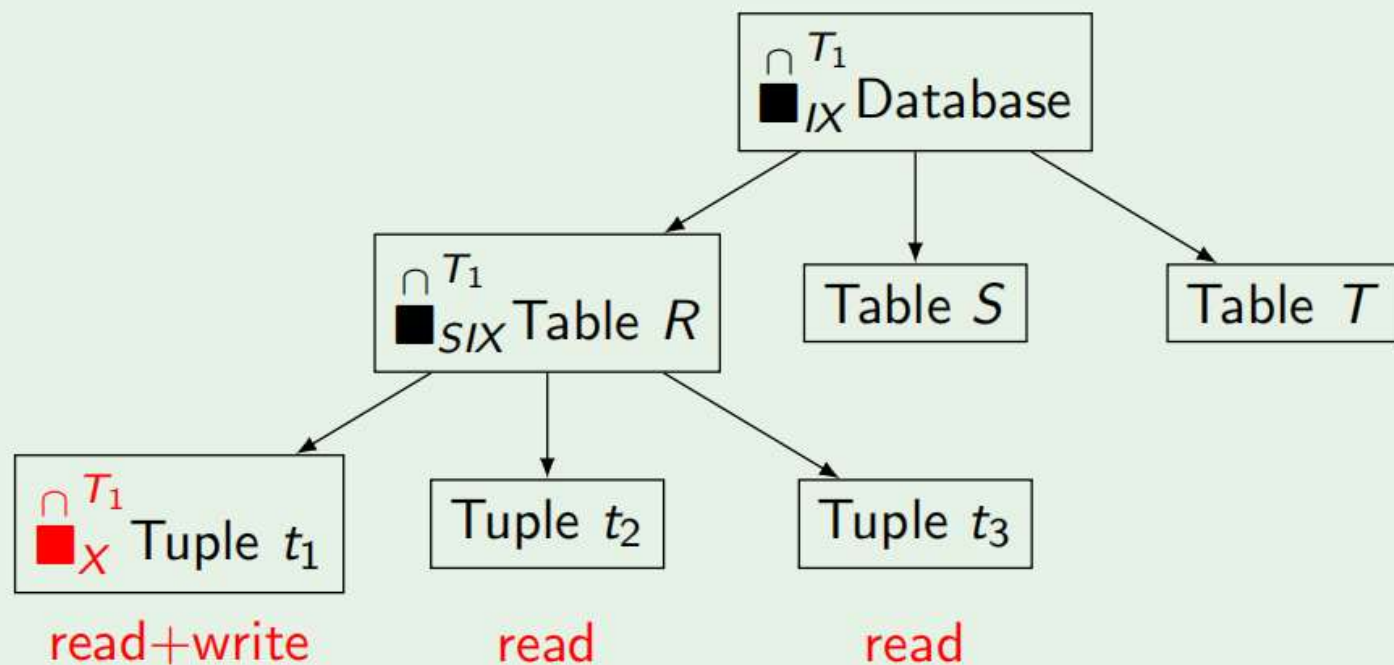
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

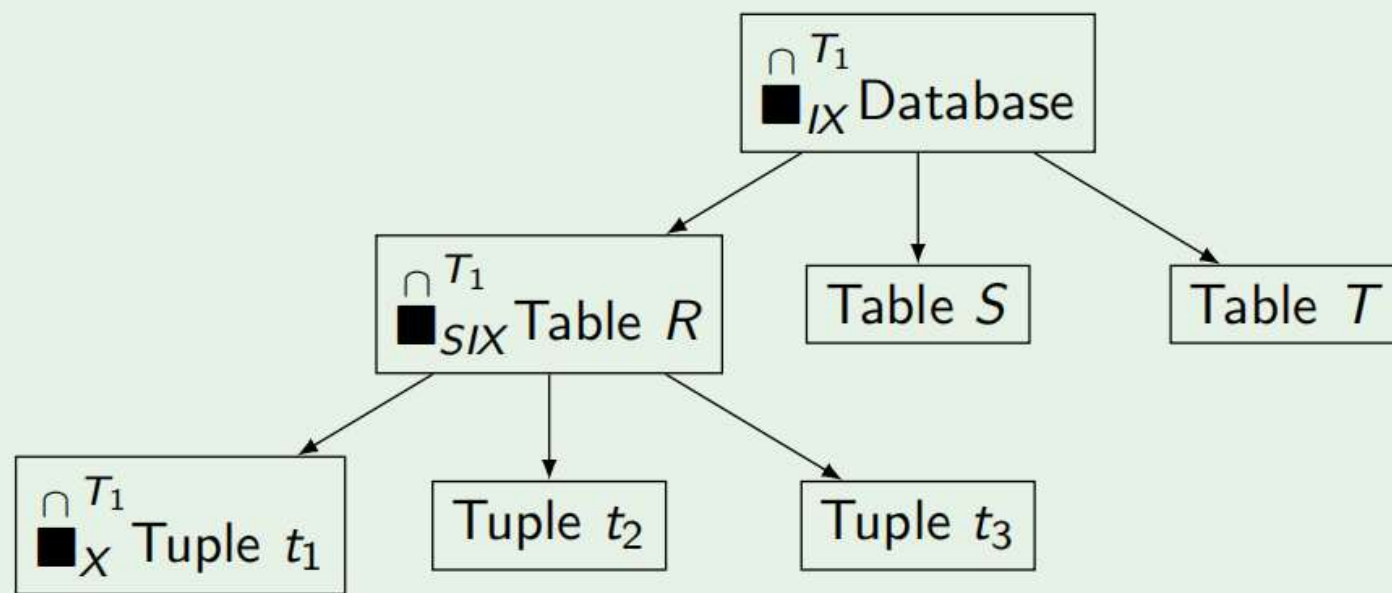
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R

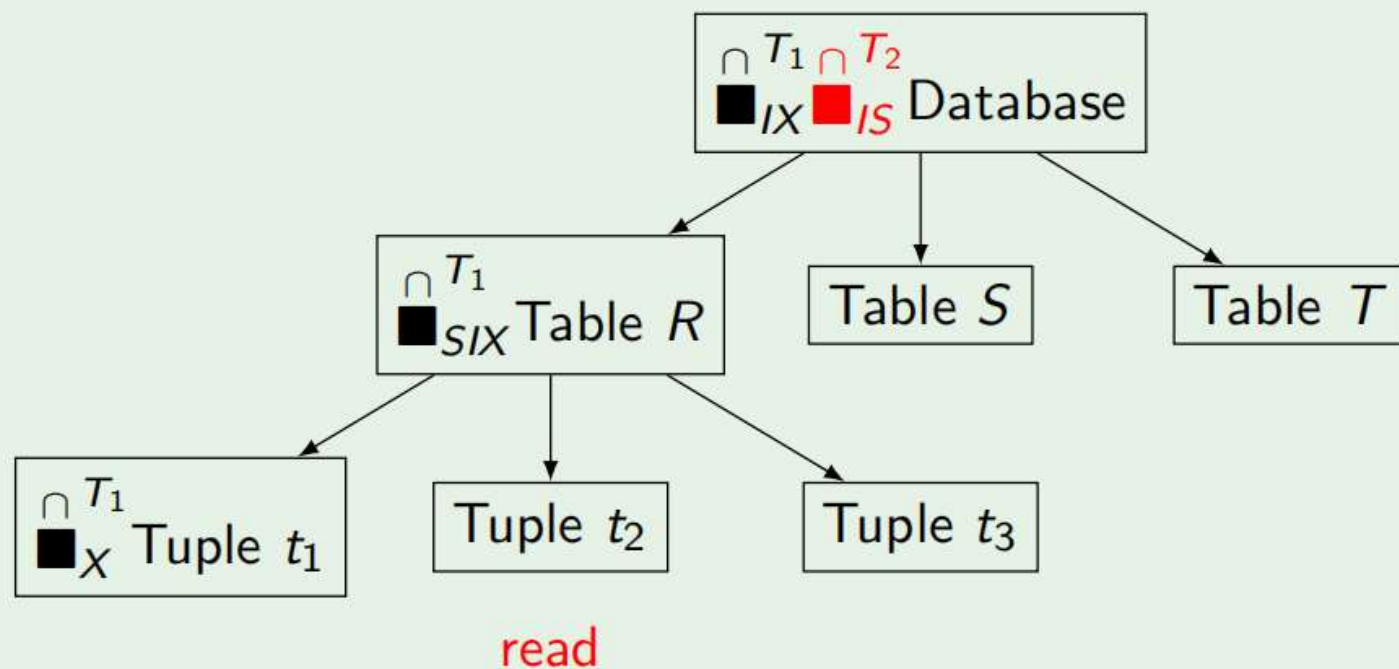


read

■ 多粒度锁协议

Example (多粒度锁协议)

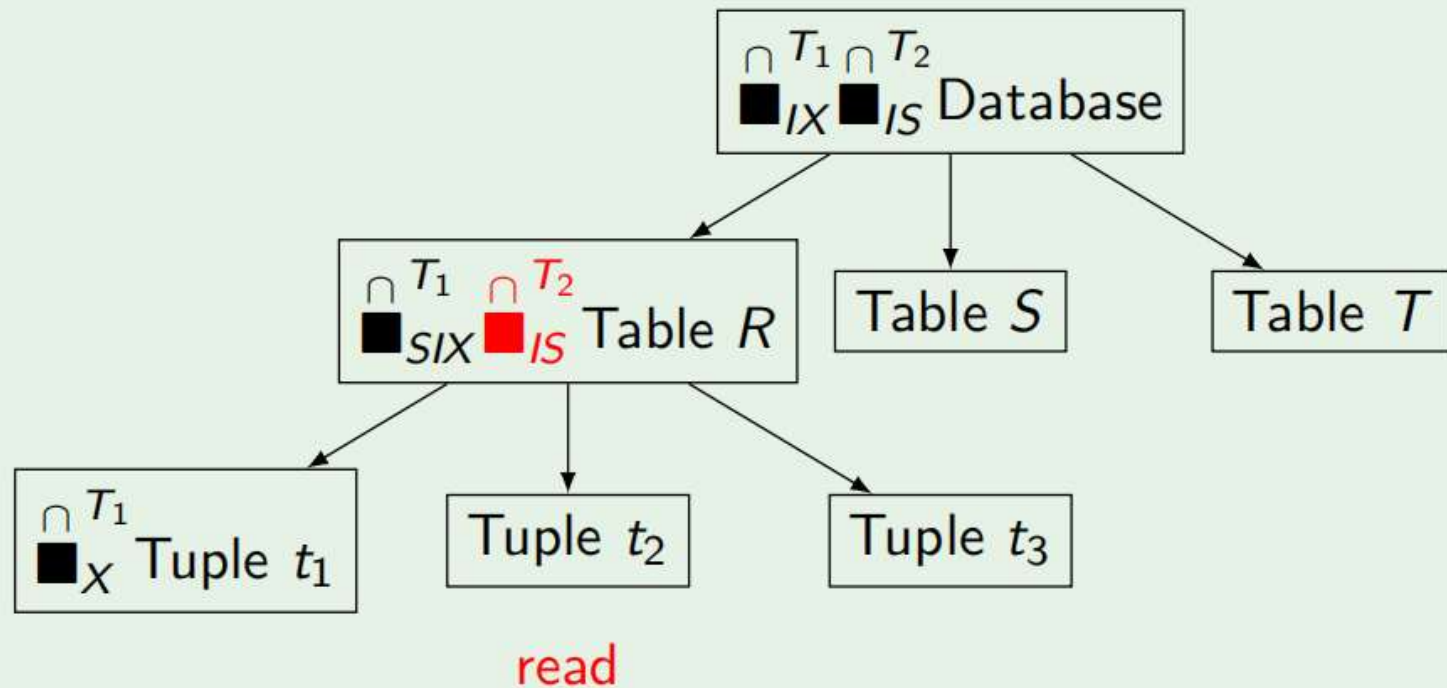
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

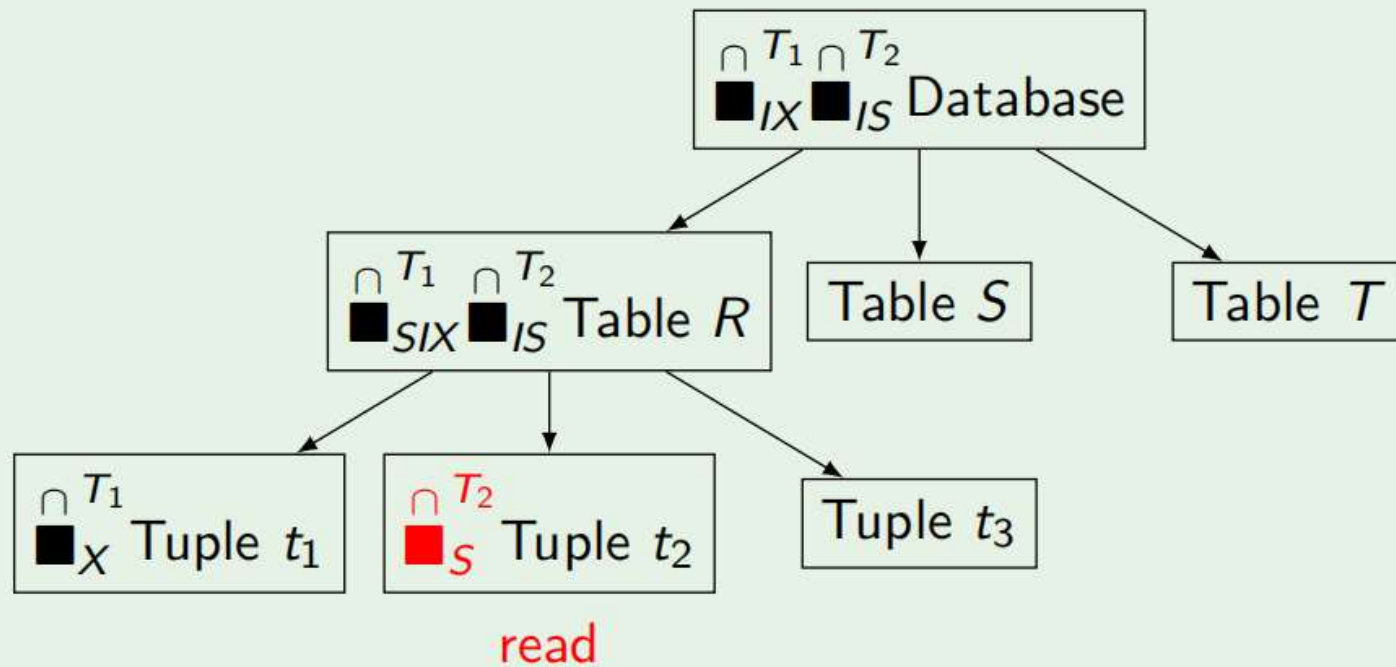
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

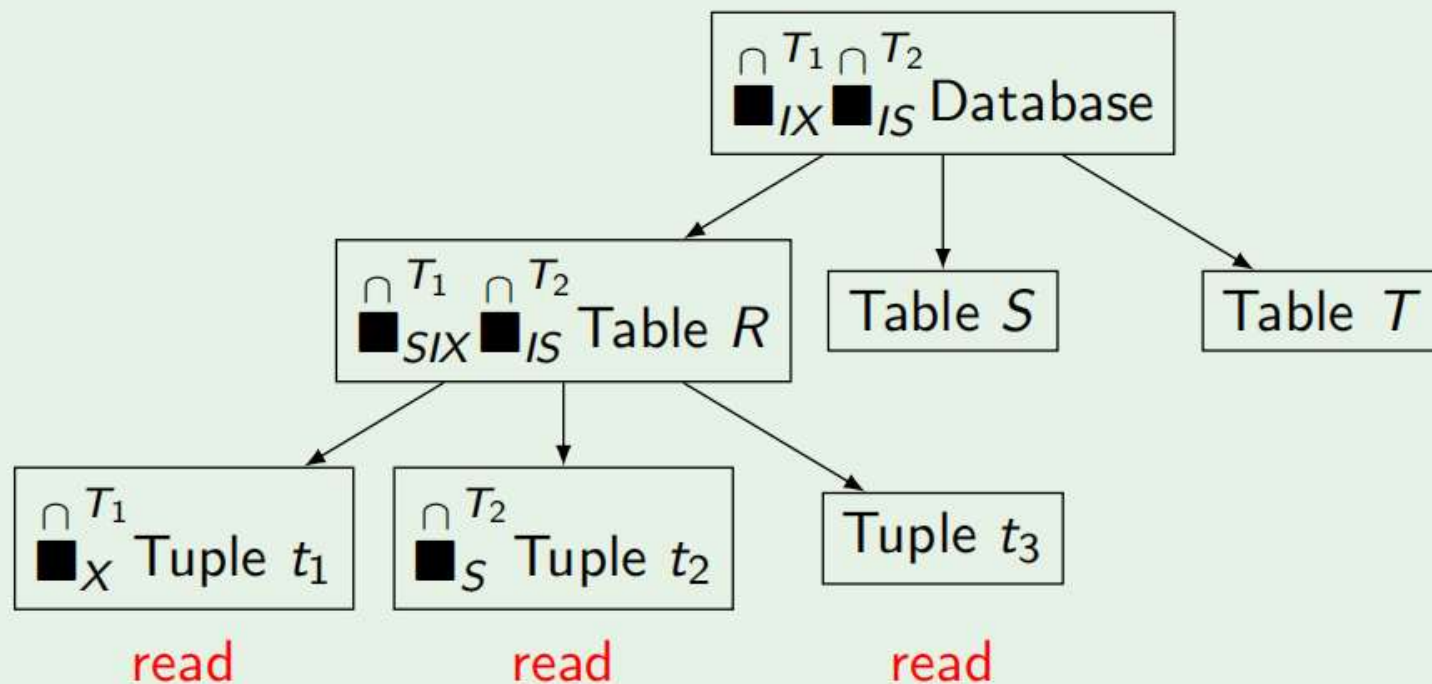
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

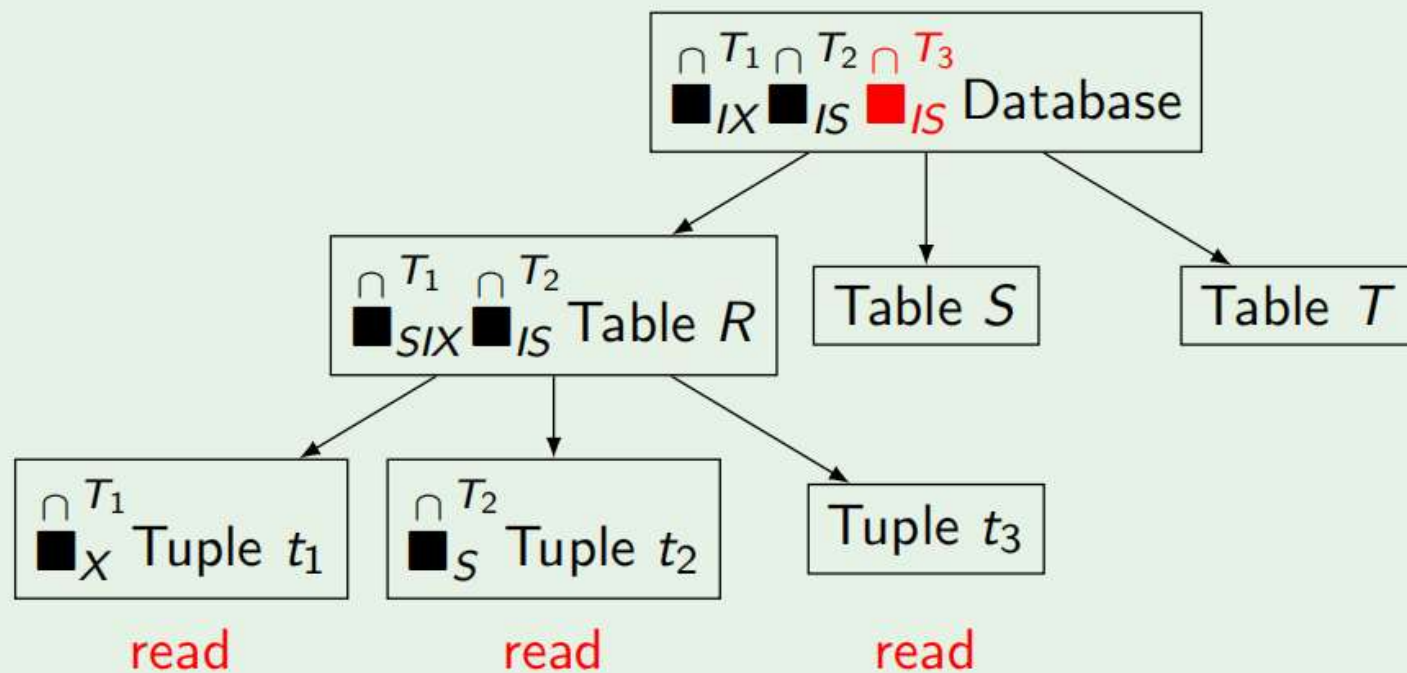
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

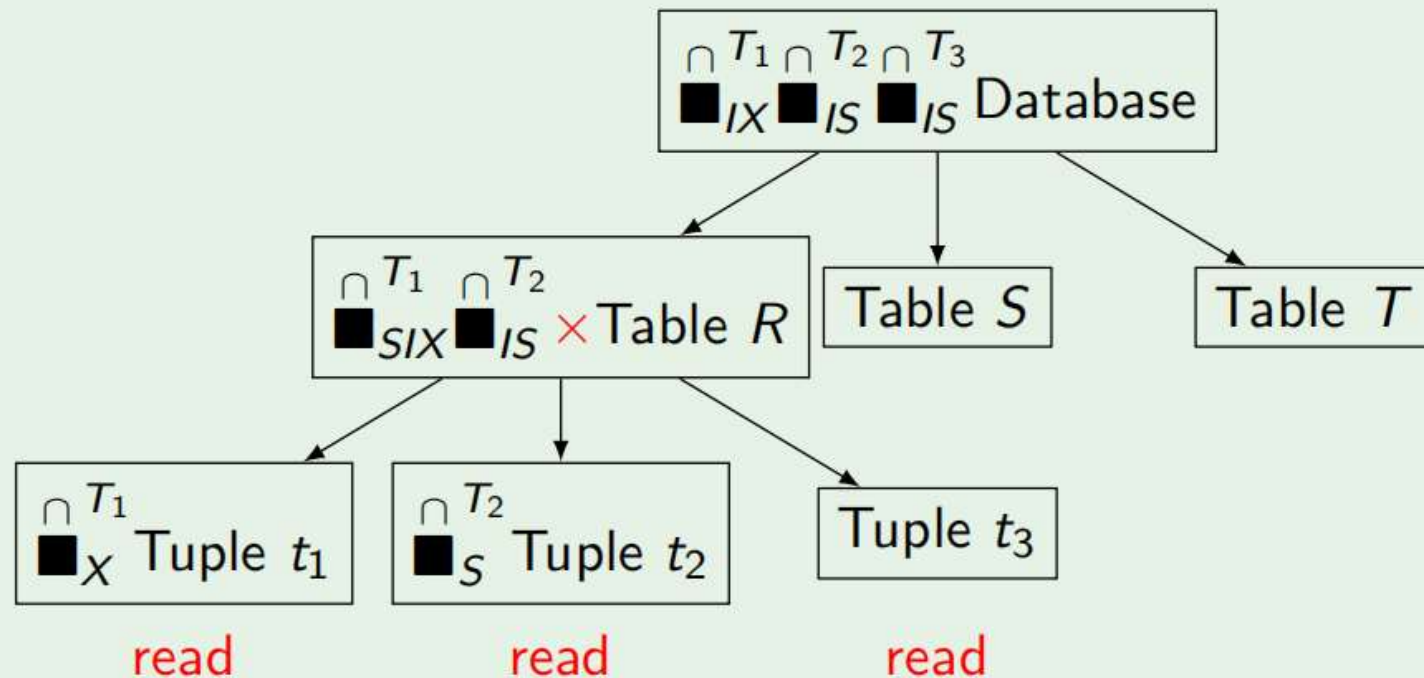
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



■ 多粒度锁协议

Example (多粒度锁协议)

- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R : 对 R 加 S 锁被拒, T_3 等待



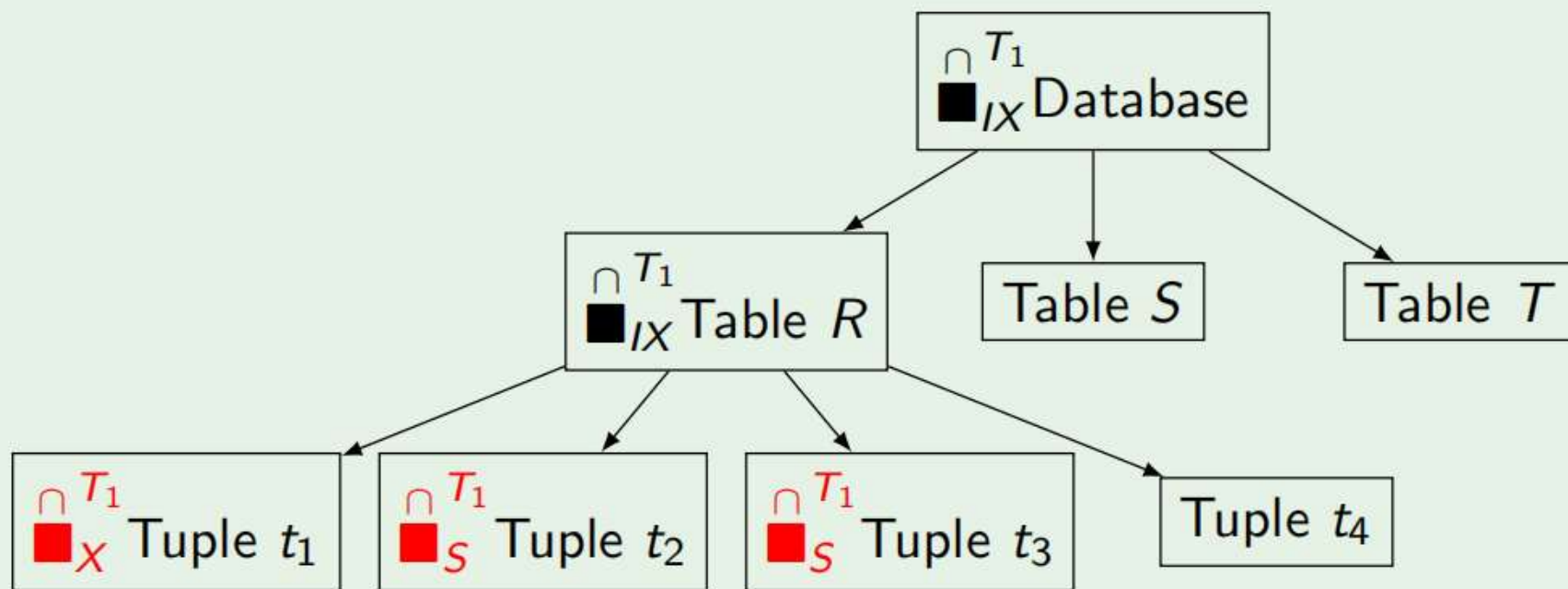
■ 锁升级

如果一个事务已经请求了大量低级别对象上的锁，则 DBMS 会动态地将这些锁升级为上一级别对象上的锁

- 减少锁的数量
- 选择合适的锁粒度

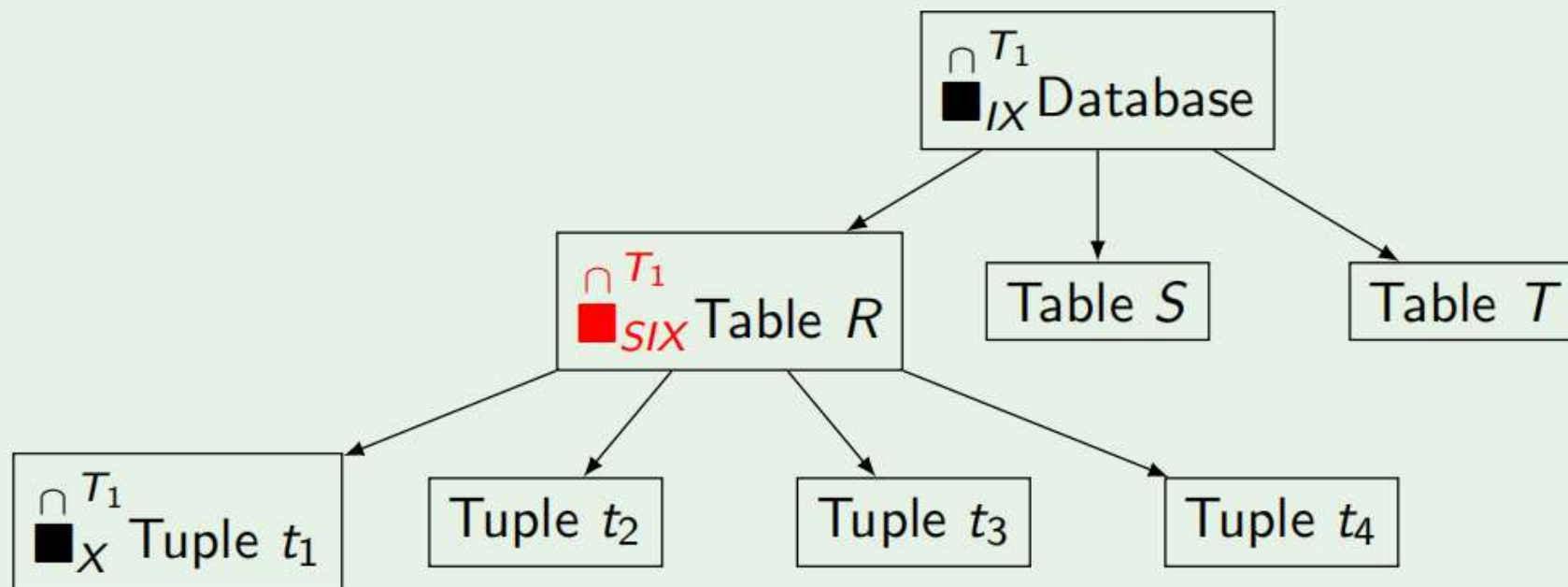
■ 锁升级

Example (锁升级)



■ 锁升级

Example (锁升级)



幻读

■ 动态数据库

前面假设事务只执行读操作和更新操作

冲突可串行化调度只在固定的数据库上保证

如果事务还执行插入或删除操作，则可能出现幻读问题

■ 幻读

Example (幻读)

T_1	T_2	T_1 Result
BEGIN	BEGIN	
SELECT MAX(val) FROM t		200
WHERE id > 1;		
	INSERT INTO t	
	VALUES (3, 300);	
	COMMIT;	
SELECT MAX(val) FROM t		300
WHERE id > 1;		
COMMIT;		

t	
id	val
1	100
2	200

原因: T_1 只能锁定 t 中 $id > 1$ 的元组, 无法给不存在的元组加锁

■ Demo

```
CREATE TABLE t (  
    id INT PRIMARY KEY,  
    val INT NOT NULL  
);
```

```
SET autocommit=OFF ;
```

■ 谓词锁

可以用谓词锁来解决幻读问题

Example (谓词锁)

T_1	T_2	Lock Actions
<code>LOCK('id > 1')</code>		Lock granted to T_1
<code>SELECT MAX(val) FROM t</code> <code>WHERE id > 1;</code>		
	<code>LOCK('id > 1')</code>	Denied
<code>SELECT MAX(val) FROM t</code> <code>WHERE id > 1;</code>		
<code>UNLOCK('id > 1')</code>		Lock released
<code>COMMIT</code>		
	<code>INSERT INTO t</code> <code>VALUES (3, 300);</code> <code>UNLOCK('id > 1')</code> <code>COMMIT</code>	Lock granted to T_2

■ Next-Key 锁

谓词锁的实现代价很高

MySQL 使用 next-key 锁

大纲

- 事务
- 并发控制
- 基于锁的并发控制
- 基于时间戳的并发控制
- 多版本并发控制 (MVCC)

■ 时间戳

DBMS 为每个事务 T_i 分配一个唯一且固定的数，作为 T_i 的时间戳，记作 $TS(T_i)$

➤ 时间戳单调递增：分配得越晚，时间戳越大

不同方法分配时间戳的时机不同

➤ 事务启动时分配，如基本 T/O

➤ 事务的验证阶段开始时分配，如乐观并发控制

■ 时间戳定序的并发控制

事务读写对象前无需加锁

根据时间戳来确定事务的可串行化调度

- 如果 $TS(T_i) < TS(T_j)$, 则 DBMS 必须确保在事务的调度所等价的串行调度中, T_i 出现在 T_j 之前

基本 T/O

■ 基本 T/O

每个对象 X 关联着 2 个时间戳

- $RTS(X)$: 成功读 X 的最晚的事务的时间戳
- $WTS(X)$: 成功写 X 的最晚的事务的时间戳

对事务的每个读写操作进行时间戳检查

- 如果一个事务准备访问一个“来自未来”的对象, 则该事务中止, 并重启为一个新事务

■ 基本 T/O 的读操作

事务 T_i 准备读对象 X

如果 $TS(T_i) < WTS(X)$, 则

➤ 中止 T_i 并重启 T_i (分配新的时间戳)

否则

➤ 允许 T_i 读 X

➤ 将 $RTS(X)$ 更新为 $\max(RTS(X), TS(T_i))$

➤ 在事务的工作区中创建 X 的局部副本, 以保证可重复读

■ 基本 T/O 的写操作

事务 T_i 准备写对象 X

如果 $TS(T_i) < RTS(X)$ 或 $TS(T_i) < WTS(X)$, 则

➤ 中止并重启 T_i

否则

➤ 允许 T_i 写 X

➤ 将 $WTS(X)$ 更新为 $\max(WTS(X), TS(T_i))$

➤ 创建 X 的局部副本

■ 基本 T/O 示例

Example (Basic T/O例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(B)$	
	BEGIN
	$r(B)$
	$w(B)$
$r(A)$	
	$r(A)$
	$w(A)$
COMMIT	COMMIT

Object	RTS	WTS
A	0	0
B	0	0

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(B)$	
	BEGIN
	$r(B)$
	$w(B)$
$r(A)$	
	$r(A)$
	$w(A)$
COMMIT	COMMIT

Object	RTS	WTS
A	0	0
B	1	0

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	0	0
B	1	0

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(B)$	
	BEGIN
	$r(B)$
	$w(B)$
$r(A)$	
	$r(A)$
	$w(A)$
COMMIT	COMMIT

Object	RTS	WTS
A	0	0
B	2	0

■ 基本 T/O 示例

Example (Basic T/O例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	0	0
B	2	0

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(B)$	
	BEGIN
	$r(B)$
	$w(B)$
$r(A)$	
	$r(A)$
	$w(A)$
COMMIT	COMMIT

Object	RTS	WTS
A	0	0
B	2	2

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	0	0
B	2	2

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	1	0
B	2	2

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	1	0
B	2	2

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	2	0
B	2	2

■ 基本 T/O 示例

Example (Basic T/O例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	2	0
B	2	2

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	2	2
B	2	2

■ 基本 T/O 示例

Example (Basic T/O 例1)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

Object	RTS	WTS
A	2	2
B	2	2

没有违反时间戳顺序， T_1 和 T_2 都可以提交

■ 基本 T/O 示例

Example (Basic T/O 例2)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

Object	RTS	WTS
A	0	0
B	0	0

■ 基本 T/O 示例

Example (Basic T/O 例2)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

Object	RTS	WTS
A	1	0
B	0	0

■ 基本 T/O 示例

Example (Basic T/O 例2)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

Object	RTS	WTS
A	1	0
B	0	0

■ 基本 T/O 示例

Example (Basic T/O 例2)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

Object	RTS	WTS
A	1	2
B	0	0

■ 基本 T/O 示例

Example (Basic T/O 例2)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

Object	RTS	WTS
A	1	2
B	0	0

■ 基本 T/O 示例

Example (Basic T/O 例2)

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

Object	RTS	WTS
A	1	2
B	0	0

T_1 不能覆盖 T_2 修改过的 A , 故 T_1 中止并重启

■ Thomas 写规则

事务 T_i 准备写对象 X

如果 $TS(T_i) < RTS(X)$, 则

- 中止 T_i 并重启 T_i (分配新的时间戳)

如果 $TS(T_i) < WTS(X)$, 则

- 忽略这个写操作并允许 T_i 继续执行(Thomas 写规则)
- 这违反了 T_i 的时间戳顺序

否则

- 允许 T_i 写 X
- 将 $WTS(X)$ 更新为 $\max(WTS(X), TS(T_i))$

■ Thomas 写规则

Example (Basic T/O例2(续))

Schedule	
T_1	T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

Database		
Object	RTS	WTS
A	1	2
B	0	0

应用Thomas写规则：忽略 T_1 对A的写操作，继续执行 T_1

■ 基本 T/O 的性质

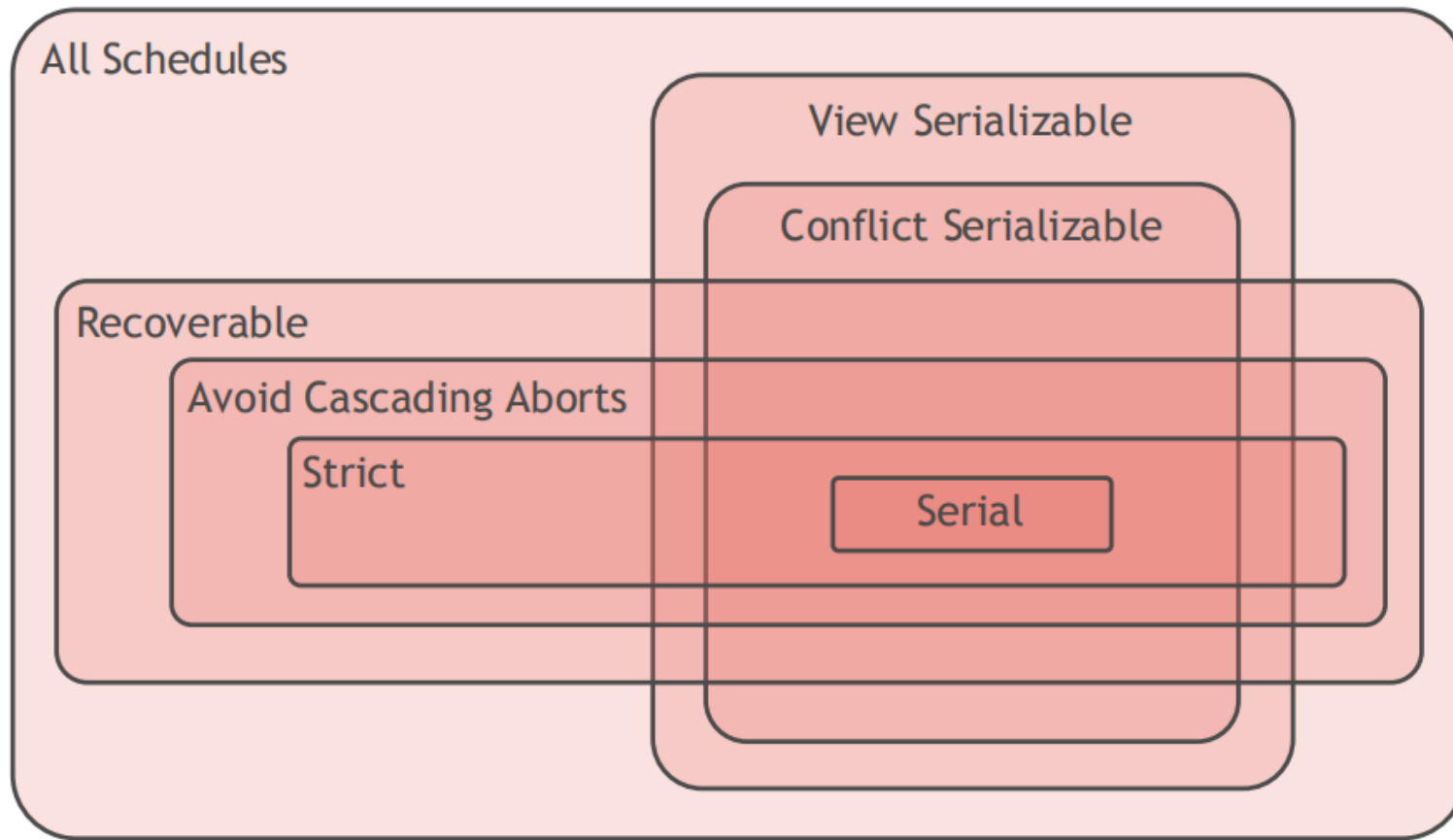
如果不使用 Thomas 写规则，则基本 T/O 生成冲突可串行化调度

- 因为任何事务都无需等待，所以无死锁
- 如果某个长事务总是和其他短事务发生冲突，则这个长事务可能会“饿死”
(不断中止并重启)

基本 T/O 可能生成不可恢复的调度

■ 可恢复调度

一个调度是可恢复调度，如果该调度中每个事务 T 都是在其它做出 T 所读到的修改的事务提交后才提交的



■ 可恢复调度

Schedule	
T_1 $TS(T_1) = 1$	T_2 $TS(T_2) = 2$
BEGIN $w(A)$	BEGIN $r(A)$ $w(B)$ COMMIT
Crash! COMMIT	

该调度不可恢复

- 在故障恢复时, DBMS undo T_1
- 因为 T_2 读了 T_1 修改过的 A , 所以必须 undo T_2
- 但 T_2 已提交, 故无法 undo T_2

■ 基本 T/O 的不足

开销大

- 必须将数据复制到事务的工作区
- 频繁更新时间戳

长事务可能会饿死

乐观并发控制

■ 乐观并发控制

基本假设

- 并发事务的冲突很少
- 大多数事务都很短

OCC 中事务的三个执行阶段

- 读阶段
- 验证阶段
- 写阶段

■ 读阶段

- DBMS 为事务创建私有工作区
- 事务读的每个对象都要复制到工作区，以保证可重复读
- 事务对对象的修改写到工作区，而不是数据库
- 将读和写分别记录到读集合和写集合中

■ 验证阶段

- 当验证阶段开始时，DBMS 为事务分配时间戳
- DBMS 对事务的写集合进行检查，判断该事务是否与其他活跃事务冲突

■ 写阶段

- 如果没有冲突，则将写集合写回数据库
- 否则，中止该事务，并重启为新事务

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
r(A)	//Read
	r(A)
	//Validate
	//Write
	COMMIT
w(A)	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read r(A)	//Read r(A)
	//Validate
	//Write
	COMMIT
w(A)	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
$r(A)$	//Read
	$r(A)$
	//Validate
	//Write
	COMMIT
$w(A)$	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	123	0

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read $r(A)$	//Read $r(A)$
	//Validate
	//Write
	COMMIT
$w(A)$	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	123	0

T_2 Workspace		
Object	Value	WTS

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read $r(A)$	//Read $r(A)$
	//Validate
	//Write
	COMMIT
$w(A)$	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	123	0

T_2 Workspace		
Object	Value	WTS
A	123	0

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read $r(A)$	//Read $r(A)$
	//Validate $TS(T_2) = 1$
	//Write COMMIT
$w(A)$ //Validate //Write COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	123	0

T_2 Workspace		
Object	Value	WTS
A	123	0

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
r(A)	//Read
	r(A)
	//Validate
	$TS(T_2) = 1$
	//Write
	COMMIT
w(A)	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	123	0

T_2 Workspace		
Object	Value	WTS
A	123	0

■ OCC 示例

Example (OCC示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
r(A)	//Read
	r(A)
	//Validate
	$TS(T_2) = 1$
	//Write
	COMMIT
w(A)	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	123	0

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
r(A)	//Read
	r(A)
	//Validate
	$TS(T_2) = 1$
	//Write
	COMMIT
w(A)	
//Validate	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	456	∞

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
r(A)	//Read
	r(A)
	//Validate
	$TS(T_2) = 1$
	//Write
	COMMIT
w(A)	
//Validate	
$TS(T_1) = 2$	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	123	0

T_1 Workspace		
Object	Value	WTS
A	456	∞

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
r(A)	//Read
	r(A)
	//Validate
	$TS(T_2) = 1$
	//Write
	COMMIT
w(A)	
//Validate	
$TS(T_1) = 2$	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	456	2

T_1 Workspace		
Object	Value	WTS
A	456	∞

■ OCC 示例

Example (OCC 示例)

Schedule	
T_1	T_2
BEGIN	BEGIN
//Read	
r(A)	//Read
	r(A)
	//Validate
	$TS(T_2) = 1$
	//Write
	COMMIT
w(A)	
//Validate	
$TS(T_1) = 2$	
//Write	
COMMIT	

Database		
Object	Value	WTS
A	456	2

■ 验证阶段

事务 T_j 进入验证阶段后

DBMS 为 T_j 分配时间戳 $TS(T_j)$

对任意 $TS(T_i) < TS(T_j)$ 的事务 T_i ，如果下面3个条件之一满足，则 T_j 与 T_i 不冲突

- 条件1: 在 T_j 开始前， T_i 已完成3个阶段
- 条件2: 在 T_j 的写阶段开始前， T_i 已完成3个阶段，且 T_i 写的对象均未被 T_j 读过，即 $WS(T_i) \cap RS(T_j) = \emptyset$
- 条件3: 在 T_j 的读阶段完成前， T_i 的读阶段已完成，且 T_i 写的对象均未被 T_j 读写过，即 $WS(T_i) \cap RS(T_j) = \emptyset$ 且 $WS(T_i) \cap WS(T_j) = \emptyset$
- 每个条件都保证了 T_j 的写对 T_i 不可见

如果所有满足 $TS(T_i) < TS(T_j)$ 的事务 T_i 与 T_j 都不冲突，则 T_j 通过验证，可以进入写阶段

■ 验证阶段 1

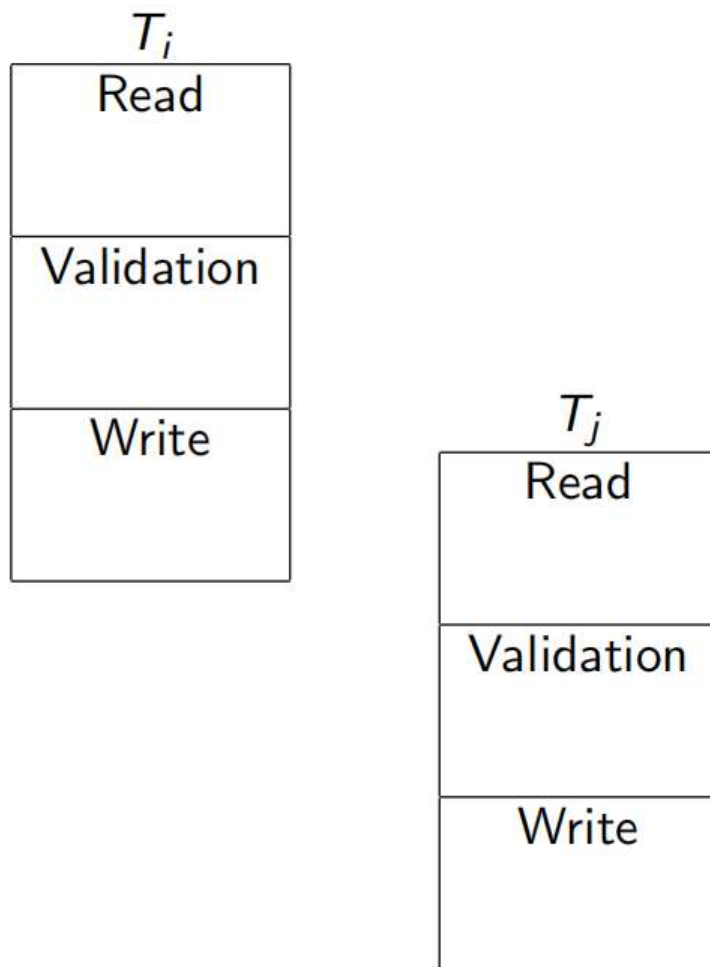
在 T_j 开始前, T_i 已完成3个阶段

Schedule	
T_i	T_j
BEGIN	
//Read	
//Validate	
//Write	
COMMIT	
	BEGIN
	//Read
	//Validate
	//Write
	COMMIT

该调度就是由时间戳确定的串行调度

■ 验证阶段 2

在 T_j 的写阶段开始前, T_i 已完成3个阶段, 且 $WS(T_i) \cap RS(T_j) = \emptyset$



Example (满足验证条件2的调度)

Schedule	
T_i	T_j
BEGIN	BEGIN
	//Read
	r(A)
//Read	w(A)
r(A)	
//Validate	
//Write	
COMMIT	
	//Validate
	//Write
	COMMIT

$WS(T_i) \cap RS(T_j) = \emptyset$, 故该调度等价于由时间戳确定的串行调度

Example (违反验证条件2的调度)

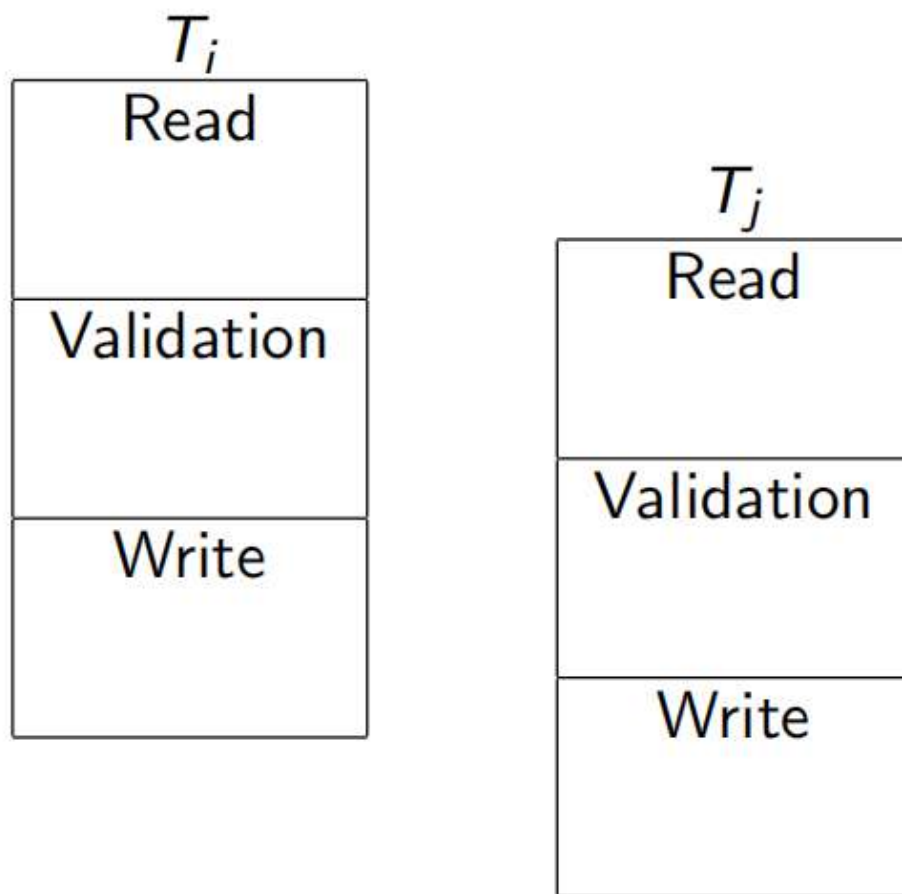
Schedule	
T_i	T_j
BEGIN	BEGIN
//Read $r(A)$ $w(A)$	
	//Read $r(A)$
//Validate //Write COMMIT	
	//Validate //Write COMMIT

$WS(T_i) \cap RS(T_j) = \{A\}$, 故该调度不等价于由时间戳确定的串行调度

- 在由时间戳确定的串行调度中, T_j 读到的 A 应当是由 T_i 写的
- 在该调度中, T_j 读到的 A 并不是 T_i 写的, 因为当 T_j 读 A 时, T_i 尚未将修改过的 A 写回数据库

■ 验证阶段 3

在 T_j 的读阶段完成前, T_i 的读阶段已完成,
且 $WS(T_i) \cap RS(T_j) = \emptyset$, $WS(T_i) \cap WS(T_j) = \emptyset$



Example (满足验证条件3的调度)

Schedule	
T_i	T_j
BEGIN	BEGIN
//Read	
r(A)	
w(A)	//Read
	r(B)
//Validate	
	w(B)
	//Validate
	//Write
	COMMIT
//Write	
COMMIT	

$WS(T_i) \cap RS(T_j) = \emptyset$ 且 $WS(T_i) \cap WS(T_j) = \emptyset$, 故该调度等价于由时间戳确定的串行调度

Example (满足验证条件3的调度)

Schedule	
T_i	T_j
BEGIN	BEGIN
//Read $r(A)$ $w(A)$	
	//Read $r(B)$
//Validate	
	$w(B)$ //Validate //Write COMMIT
//Write COMMIT	

$WS(T_i) \cap RS(T_j) = \emptyset$ 且 $WS(T_i) \cap WS(T_j) = \emptyset$, 故该调度等价于由时间戳确定的串行调度

Example (违反验证条件3的调度)

Schedule	
T_i	T_j
BEGIN	BEGIN
//Read $r(A)$ $w(A)$	
	//Read $r(A)$
//Validate	
	$r(B)$ //Validate //Write COMMIT
//Write COMMIT	

$WS(T_i) \cap RS(T_j) = \{A\}$, 故该调度不等价于由时间戳确定的串行调度

- 在由时间戳确定的串行调度中, T_j 读到的 A 应当是由 T_i 写的
- 在该调度中, T_j 读到的 A 并不是 T_i 写的, 因为当 T_j 读 A 时, T_i 尚未将修改过的 A 写回数据库

■ 串行验证

在临界区内执行事务的验证阶段和写阶段

- 任何时刻只有一个事务处于验证/写阶段
- 当一个事务正在验证时，其他事务不得提交
- 如果一个事务已经完成验证阶段，则只有在其写阶段完成后，其他事务才能开始验证

■ OCC 的特性

在并发事务冲突很少的情况下，OCC 很有效

- 例如，事务都是只读的
- 例如，事务访问的数据子集互不相交

■ OCC 的不足

- 将数据复制到工作区的开销很大
- 验证阶段和写阶段是瓶颈
- OCC 比 2PL 的事务中止代价高，因为中止事务的读阶段已经完成

■ 基本 T/O 和 OCC 的区别

基本 T/O

- 基本 T/O 在执行事务的每个读写操作前都要进行时间戳检查
- 事务的运行时代价比较高

OCC

- OCC在执行事务的读写操作前不需要进行时间戳检查，只需在验证阶段进行检查
- 事务中止的代价比较高

大纲

- 事务
- 并发控制
- 基于锁的并发控制
- 基于时间戳的并发控制
- 多版本并发控制 (MVCC)

■ 多版本并发控制

DBMS 为数据库中每个(逻辑)对象维护多个(物理)版本

- 当事务写一个对象时, DBMS 为该对象创建一个新的版本
- 当事务读一个对象时, 它读的是该事务启动时已存在的由已提交事务写入的最新版本

■ MVCC 的特点

- 写不阻塞读
- 读不阻塞写

只读的事务直接在一致性快照上读，无需加锁

■ MVCC 示例

Example (MVCC 示例1)

Schedule	
T_1	T_2
BEGIN	
r(A)	
	BEGIN
	w(A)
r(A)	
COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active

■ MVCC 示例

Example (MVCC 示例1)

Schedule	
T_1	T_2
BEGIN $r(A) \leftarrow A_0$	BEGIN $w(A)$
$r(A)$ COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active

■ MVCC 示例

Example (MVCC 示例1)

Schedule	
T_1	T_2
BEGIN $r(A)$	
	BEGIN $w(A)$
$r(A)$ COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例1)

Schedule	
T_1	T_2
BEGIN $r(A)$	BEGIN $w(A)$
$r(A)$ COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	2
A_1	456	2	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例1)

Schedule	
T_1	T_2
BEGIN	BEGIN
$r(A)$	
$r(A) \leftarrow A_0$	
COMMIT	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	2
A_1	456	2	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例1)

Schedule	
T_1	T_2
BEGIN	BEGIN
r(A)	
r(A)	
COMMIT	w(A)
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	2
A_1	456	2	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Committed
T_2	2	Committed

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	
r(A)	
w(A)	BEGIN
	r(A)
	w(A)
r(A)	
COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	BEGIN
$r(A) \leftarrow A_0$	
$w(A)$	
$r(A)$	$r(A)$
COMMIT	$w(A)$
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	BEGIN
r(A)	
w(A)	
r(A)	r(A)
COMMIT	w(A)
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	
r(A)	
w(A)	BEGIN
	r(A)
	w(A)
r(A)	
COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	BEGIN
r(A)	
w(A)	
	$r(A) \leftarrow A_0$
r(A)	w(A)
COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	BEGIN
$r(A)$	
$w(A)$	
$r(A)$	$w(A)$ Stall...
COMMIT	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A)
	w(A) Stall...
$r(A) \leftarrow A_1$	
COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Active
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A)
	w(A) Stall...
r(A)	
COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Committed
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	
r(A)	
w(A)	BEGIN
	r(A)
	w(A)
r(A)	
COMMIT	
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	2
A_2	789	2	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Committed
T_2	2	Active

■ MVCC 示例

Example (MVCC 示例2)

Schedule	
T_1	T_2
BEGIN	BEGIN
r(A)	
w(A)	
r(A)	r(A)
COMMIT	w(A)
	COMMIT

Database			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	2
A_2	789	2	-

Txn Status Table		
TxnId	TS	Status
T_1	1	Committed
T_2	2	Committed

■ 并发控制协议

- 写不阻塞读
- 读不阻塞写
- 写阻塞写

采用并发控制协议解决写写冲突问题

- 时间戳定序并发控制
- 乐观并发控制
- 两阶段锁协议

大纲

- 事务
- 并发控制
- 基于锁的并发控制
- 基于时间戳的并发控制
- 多版本并发控制 (MVCC)